

Deep Learning Latency Attacks and Defenses: A Cross-Domain Survey of Availability Threats

ZONGHUA GU, Hofstra University, USA

ZEYU GAO, Umeå University, Sweden

AMIN SAREMI, Umeå University, Sweden

SAMARJIT CHAKRABORTY, University of North Carolina at Chapel Hill, USA

Adversarial machine learning has historically focused on *integrity*: protecting models against attacks that cause them to produce incorrect outputs. A complementary and increasingly consequential axis is *availability*, where the attacker’s objective is not necessarily to corrupt a prediction but to make the system violate its latency, energy, throughput, or resource constraints. *Latency attacks* (also termed energy-latency or resource-consumption attacks) are availability attacks that increase inference-time work, energy consumption, or response time, potentially causing deadline misses, throughput collapse, or resource exhaustion in a deployed system such as a vehicle controller, an interactive service, or a battery-powered sensor. Some preserve the nominal prediction or reported task metric, whereas others also degrade output quality or task completion. This type of attack is especially problematic for machine-learning systems deployed on embedded platforms with limited computational, memory, and battery resources that operate under tight real-time deadlines.

This survey synthesizes a literature that has, until now, been fragmented across distinct communities. The survey spans three broad settings: perception pipelines, including physical-world attacks on autonomous-driving detection and tracking; input-adaptive neural inference, including sponge examples, multi-exit networks, translation, and captioning; and autoregressive or agentic systems, including recent output-length, verbose-image, and reasoning denial-of-service attacks against large language models (LLMs), vision-language models (VLMs), mixture-of-experts (MoE) systems, and tool-using agents.

A distinguishing feature of this survey is its organizing principle: rather than grouping attacks by individual formulations, we organize the literature around exploited computational bottlenecks and their roles in end-to-end AI systems. Specifically, we structure the field along two complementary axes: the *exploited bottleneck* (post-processing, temporal state, sensor/cooperative fusion, hardware, dynamic-computation control flow, autoregressive generation, and training-time poisoning) and the affected *model class* (CNN detectors, dynamic/adaptive networks, transformers, LLMs, and multimodal generators). We present a unified threat-model taxonomy, consolidated quantitative comparisons of reported latency, energy, work, and output-amplification measurements, and a structured review of the nascent defense landscape, including budget enforcement, robust/adaptive training, input transformation, architectural sparsity, runtime monitoring, and system-level serving control.

Finally, we identify the open challenges that will shape the next phase of this field, including standardized evaluation, physically realizable attacks, inference-time compute scaling, and whole-system availability as an emerging systems-level security objective. A companion website is available at <https://github.com/guzonghua/awesome-latency-attacks/>.

Additional Key Words and Phrases: availability attacks, energy-latency attacks, sponge examples, inference latency, denial-of-service, object detection, non-maximum suppression, multi-exit networks, large language models, vision-language models, mixture of experts, real-time systems, adversarial machine learning

CONTENTS

Abstract	1
Contents	1
1 Introduction	3

Authors’ Contact Information: Zonghua Gu, Hofstra University, Hempstead, NY, USA, zonghua.gu@hofstra.edu; Zeyu Gao, Umeå University, Umeå, Sweden, zeyu.gao@umu.se; Amin Saremi, Umeå University, Umeå, Sweden, amin.saremi@umu.se; Samarjit Chakraborty, University of North Carolina at Chapel Hill, Chapel Hill, NC, USA, samarjit@cs.unc.edu.

50	1.1	Why latency attacks matter now	3
51	1.2	Scope and contributions	4
52	2	Background and Definitions	5
53	2.1	Integrity versus availability	5
54	2.2	Where latency comes from	5
55	2.3	The unifying principle: intermediate-work amplification	6
56	2.4	Related surveys	7
57	2.5	Metrics and a working formalization	8
58	3	Threat Models	11
59	3.1	Attacker interface	12
60	3.2	Attacker knowledge	12
61	4	Latency Attacks on Object Detection and Perception	13
62	4.1	Non-maximum suppression as a bottleneck	13
63	4.2	Whole-pipeline and temporal-state attacks	13
64	4.3	Physical-world latency attacks	14
65	4.4	Beyond cameras: LiDAR and cooperative perception	14
66	4.5	Embodied and vision-language-action availability attacks	14
67	5	Neural Inference Work-Inflation Attacks	16
68	5.1	Inference-time sponge examples	16
69	5.2	Sponge poisoning: moving the attack to training	16
70	5.3	Sparsity and spiking networks	17
71	5.4	Dynamic and adaptive networks	17
72	6	Autoregressive and Serving-Side Work Inflation	18
73	6.1	Sequence generation and decoder-length inflation	18
74	6.2	LLM and VLM resource-consumption attacks	19
75	7	Training-Time Persistence, Hardware, and Timing Channels	21
76	8	Defenses	23
77	8.1	Budget enforcement and compute caps	23
78	8.2	Robust and adaptive training	24
79	8.3	Input transformation and purification	24
80	8.4	Bounded and constant-time execution	25
81	8.5	Architectural sparsity and efficiency reduction	25
82	8.6	Runtime monitoring and adaptive inference	25
83	8.7	System-level and serving control	25
84	9	Cross-Domain Synthesis and Evaluation	26
85	9.1	Evaluation and benchmarking	28
86	10	Emerging Surfaces and Open Challenges	30
87	10.1	NMS-free detectors and moving bottlenecks	30
88	10.2	Cross-cutting open challenges	31
89	10.3	Toward whole-system availability	33
90	11	Conclusion	34
91		References	35
92	A	Glossary of Metrics and Terms	42
93	B	Catalog of Latency, Energy, and Timing Attacks and Defenses	42
94	C	Attack-Mapping Matrix	47

1 Introduction

Deep neural networks are increasingly deployed in settings where *when* a prediction arrives is as important as *whether* it is correct. An autonomous vehicle that perceives a pedestrian one second late may, depending on its planning horizon and fallback behavior, have effectively not perceived it in time to act; an interactive assistant that takes thirty seconds to begin responding is unusable; a battery-powered sensor whose model suddenly draws ten times its budgeted energy will exhaust itself before its mission completes. These deployments expose an attack surface that classical adversarial machine learning largely ignored.

Most adversarial-example research targets *integrity*: an attacker perturbs an input so that the model misclassifies it [32, 104]. *Availability* attacks pursue a different goal. Rather than primarily seeking an incorrect answer, the attacker seeks to inflate the cost or delay of producing one—measured in floating-point operations, energy, or latency—so the system cannot respond “within a reasonable time” [9]. We use *latency attack* as an umbrella term for this family, which also appears in the literature under the names energy-latency attack, sponge attack, slowdown attack, efficiency-degradation attack, and resource-consumption denial-of-service. A recurring mechanism is the deliberate steering of a model, or its host system, toward unusually expensive or configured maximum-cost behavior. We reserve the term *worst case* for a proven or exhaustively characterized maximum, and otherwise describe attacks as inducing high-cost execution paths or near-worst observed behavior, since most cited studies empirically find high-cost inputs rather than establish a true maximum.

The overall organization of this survey is summarized in Figure 1.

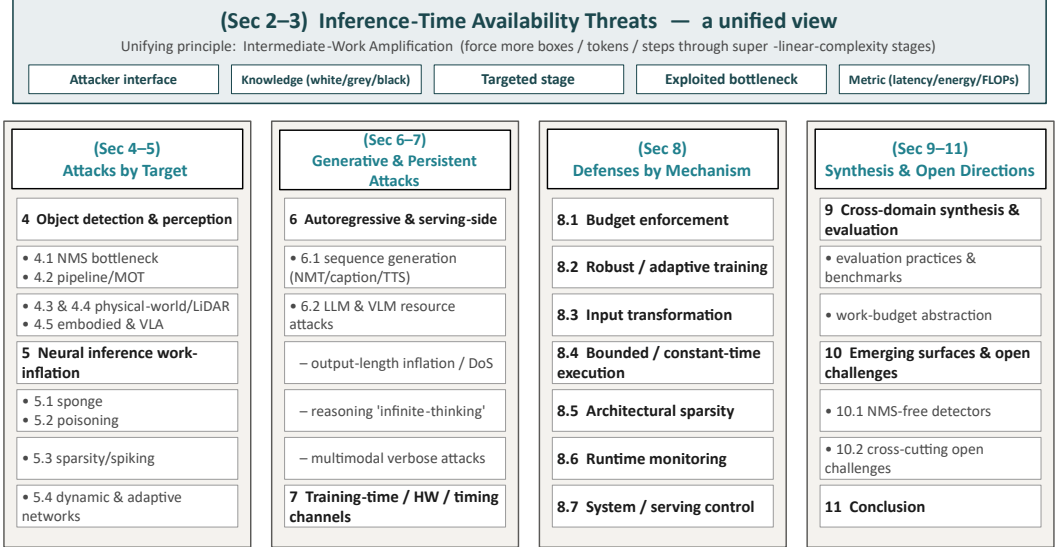


Fig. 1. Overview of the survey structure.

1.1 Why latency attacks matter now

Although many works surveyed in this paper are recent, the rapid expansion of latency attacks across perception, adaptive inference, and foundation models motivates a unified survey at this stage. Three trends have converged to make latency attacks a first-class concern. First, *real-time autonomy*

couples perception latency directly to physical safety: a delayed detection cascades into delayed planning and control [72]. Second, the field has embraced *input-adaptive efficiency*—early-exit networks, token pruning, skimming transformers, and mixture-of-experts routing—which replace fixed-cost execution with input-adaptive execution whose average cost is lower than its configured maximum. An availability attacker may revoke these savings for adversarially chosen inputs by steering them onto expensive execution paths, and under sustained load may substantially reduce system-level efficiency, in some cases approaching or exceeding the cost of the corresponding static model [15, 42]. Third, the rise of *autoregressive generation* in LLMs and VLMs means that a small input can induce an output many times longer than the benign baseline—potentially reaching the configured token, context, or execution limit—and the per-query compute cost is now an attacker-controllable variable [20, 29]. The move to inference-time “thinking” in reasoning models multiplies this leverage: a single innocuous-looking prompt has been reported to trigger tens of thousands of reasoning tokens [66].

1.2 Scope and contributions

Prior surveys treat pieces of this landscape in isolation. The closest published survey is by Brachemi Meftah et al. [7], which provides a comprehensive review of energy-latency attacks on regular, input-adaptive, and decoder-based models. However, its primary focus remains energy-latency attacks at the model level. Recent developments—including reasoning-model denial-of-service, LLM/VLM serving attacks, agentic systems, and embodied AI—substantially broaden the attack landscape and motivate a new cross-domain synthesis. The autonomous-driving security literature, meanwhile, has produced a rich set of *physically realizable* latency attacks that are rarely discussed alongside the general deep-learning work. This survey deliberately bridges both. We make the following contributions:

- **A unified taxonomy** that organizes latency attacks by exploited bottleneck and by affected model class, spanning object detection, dynamic networks, transformers, LLMs, and multimodal generators.
- **A unifying principle**—*intermediate-work amplification* (Section 2.3)—that explains a large and important subset of latency attacks through a shared mechanism (hardware fault injection and timing leakage require separate treatment) and motivates a cross-domain *work-budget* defense abstraction.
- **A threat-model framework** (Section 3) that classifies the attacker interface, knowledge, and targeted pipeline stage, making otherwise incomparable works commensurable.
- **Consolidated quantitative comparisons** (Tables 3 and 4) of reported latency, energy, computational-work, and output-length measurements, with explicit notes on metric incompatibility.
- **A structured defense review** (Section 8 and Table 6) organized by control mechanism and evidence class, exposing which point of the compute pathway each defense actually bounds.
- **A cross-domain synthesis and evaluation discussion** (Section 9) analyzing why most existing perception- and serving-side defenses do not transfer directly, while identifying work-budget control as a possible cross-domain abstraction, and including a review of fragmented evaluation practice (Section 9.1).
- **A forward-looking analysis** (Section 10, including the NMS-free detector discussion in Section 10.1) of emerging attack surfaces and the open challenges that define the next phase of this research.

The remainder of this survey develops this perspective in two steps. We first show that many seemingly unrelated latency attacks are instances of intermediate-work amplification across different AI architectures, and then argue that bounding intermediate work through work-budget enforcement provides a common systems-level defense principle.

2 Background and Definitions

2.1 Integrity versus availability

Deep learning is now pervasively deployed on resource-constrained embedded and edge platforms, where inference latency, energy, and throughput are first-class design constraints rather than afterthoughts [52, 69, 75]. It is precisely in these latency- and energy-sensitive regimes that availability attacks become consequential. We adopt the NIST taxonomy of adversarial machine learning, which distinguishes integrity, availability, and confidentiality violations [104]. Integrity attacks aim to change the model’s output; availability attacks aim to degrade its usefulness or serviceability. Latency attacks are availability attacks whose specific lever is computational cost. Crucially, a latency attack may leave the prediction *unchanged*—often by design—so a monitor based only on a preserved task-quality metric (BLEU for translation, mAP for detection, perplexity for text) may not flag an attack whose principal effect is increased execution cost [13, 94], unless it also observes timing, repetition, truncation, confidence, or distribution-shift signals.

Relation to classical denial-of-service. For readers coming from network security, it helps to contrast latency attacks with classical denial-of-service (DoS). A classical network DoS exhausts a shared resource—bandwidth, connection tables, or server sockets—typically through *volume*: many requests, oversized payloads, or malformed packets, mitigated by rate limiting, filtering, and over-provisioning at the network and transport layers. Latency attacks differ in three ways. First, the exhausted resource is *model computation*—FLOPs, accelerator time, memory bandwidth, energy, or generated tokens—rather than network capacity. Second, and most importantly, the leverage comes not from volume but from *per-request work amplification*: a single, well-formed, benign-looking input of ordinary size can induce disproportionate internal computation, so the attack often passes request-rate and payload-size filters that stop volumetric DoS. Third, the attack surface is the model’s own input-adaptive computation (post-processing, conditional execution, autoregressive generation), which classical DoS defenses do not model. In this sense a latency attack is an *algorithmic-complexity* DoS specialized to learned, input-adaptive inference: the closest classical analogue is a hash-collision or regular-expression complexity attack, but here the expensive path is induced inside a neural pipeline and can preserve the nominal output while still consuming the budget. This is why the defenses we survey (Section 8) live at the model and serving layers—bounding admitted work per request—rather than at the network perimeter.

2.2 Where latency comes from

A modern inference pipeline contains many cost centers, and latency attacks succeed by driving one or more toward unusually expensive or configured maximum-cost behavior:

Data-dependent post-processing. Operations whose cost depends on the *content*, not just the shape, of intermediate tensors. The canonical example is non-maximum suppression (NMS) in object detectors, whose cost grows with the number of candidate boxes [94].

Activation sparsity. Some accelerators and sparse kernels exploit structured or unstructured zero activations to skip operations or memory transfers—the realized energy and latency benefit depends on hardware and kernel support, and commodity dense GPU kernels generally do not skip arbitrary zero activations automatically—and approximate-computing

techniques trade accuracy for energy on the edge [33]. Inputs that minimize sparsity force maximum multiply-accumulate work and energy [77, 95].

Conditional computation. Early-exit networks, token-pruning vision transformers, skimming language models, and mixture-of-experts routers all decide *at runtime* how much computation to perform [23, 87, 102]; the same input-adaptive model-selection strategies that save average-case energy on embedded systems [75] create the worst-case headroom an attacker can reclaim. Each decision point is an attack target.

Autoregressive generation. Sequence models emit one token at a time until an end-of-sequence (EOS) signal. The number of tokens—and thus total cost—is determined by the model’s own outputs, which the attacker can steer [13, 29].

Temporal and multi-agent state. Pipelines that maintain state across frames (multi-object tracking) or aggregate across agents (cooperative perception) add stages whose cost depends on the number of tracked or fused entities [48, 106].

Hardware and memory. Memory access patterns, DRAM bandwidth, and even induced bit-flips can be manipulated to inflate latency below the algorithmic level [79, 96].

2.3 The unifying principle: intermediate-work amplification

A mechanism recurs across most of the attacks we survey, which we call *intermediate-work amplification*. The adversary engineers a situation in which some internal boundary of the computation receives more objects, tokens, attention computations, or reasoning steps than a benign input of the same nominal class would generate. In object detection, those objects are phantom bounding boxes that flood NMS or multi-object tracking (MOT). In transformer systems, the inflated intermediates may be retained tokens, KV-cache entries, selected expert assignments, or autoregressive decoding steps. In autoregressive decoding specifically, they are extra generated tokens; in reasoning models, extra chain-of-thought steps. The leverage differs by mechanism, and it is worth being precise rather than asserting a single asymptotic law. NMS is roughly quadratic in the number of retained candidate boxes in common iterative implementations and in the worst case (practical runtime also depends on top- k filtering, confidence thresholds, class-wise processing, and kernel vectorization); self-attention is quadratic in sequence length for a full attention matrix, though the marginal per-step cost under a KV cache is closer to linear in context length and is often dominated by memory traffic; autoregressive generation is linear in the number of output steps, so its leverage comes from inflating the *step count* rather than from super-linear per-step work. What these cases share is not a common complexity class but a common *shape*: a stage whose cost rises with a data-dependent intermediate count that the attacker can inflate while sometimes preserving the nominal output or the source paper’s reported task metric (though repetitive generation, task failure, halted actions, or truncated outputs need not preserve either). This shared structure appears across perception pipelines and LLM/VLM-serving domains. It also suggests a candidate cross-domain defense—a *work budget* that caps the admitted intermediate count at a stage boundary (Section 9)—though, as we discuss there, such caps carry their own utility and fairness costs and are not a universal remedy.

This perspective provides the central organizing principle of this survey. Although latency attacks have emerged independently across object detection, adaptive neural networks, autoregressive language generation, reasoning models, and agentic systems, many can be understood through the common lens of intermediate-work amplification: rather than directly changing a model’s prediction, the attacker increases the amount of intermediate computation that the system performs. The specific intermediate differs across domains—for example, candidate boxes in object detection, tracked objects in multi-object tracking, retained tokens in transformers, activated experts in

mixture-of-experts models, generated tokens in LLMs, or reasoning steps in large reasoning models—but the underlying objective is the same: to amplify computational work and thereby degrade system availability. This shared viewpoint naturally motivates a common defensive principle. Instead of attempting to detect every attack individually, systems can explicitly bound the amount of intermediate work admitted to computationally expensive stages through work-budget enforcement, such as limiting candidate boxes, generated tokens, or association operations. We revisit this abstraction throughout the survey as a unifying perspective for both attacks and defenses, while noting where it does not apply (for example, hardware fault injection and timing side channels; Section 7).

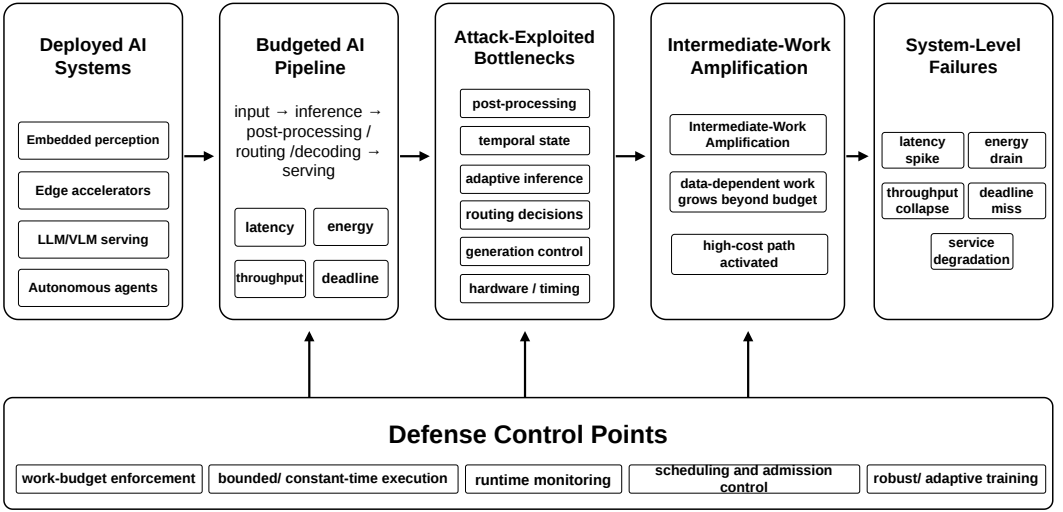


Fig. 2. Overview of latency attacks as system-level availability threats.

Figure 2 presents an overview of latency attacks as system-level availability threats. Deployed AI systems operate under latency, energy, throughput, and deadline budgets. Attackers exploit data-dependent bottlenecks to amplify intermediate work and push execution toward high-cost paths, causing latency spikes, energy drain, throughput collapse, missed deadlines, or service degradation. The same data-dependent runtime can additionally be observed as an *adjacent timing side channel*—drawn as a separate branch in the figure—which reveals internal state rather than degrading availability; because an attacker need not amplify work to exploit a timing channel, we keep it distinct from the availability consequences above (Section 3). Defenses restore bounded and predictable computation through budget enforcement, monitoring, scheduling, and bounded execution.

2.4 Related surveys

Adversarial-ML surveys overwhelmingly emphasize *evasion*—perturbations that cause misclassification—and treat the availability axis as peripheral [1, 34, 62]. Surveys of adversarial attacks on vision-language models catalog jailbreak, backdoor, and patch threats but largely omit output-length and resource-consumption attacks [18, 27, 43]. LLM-security surveys catalog serving-layer availability attacks but do not connect them to the perception-stack latency literature [120]. Earlier work by Kianpour and Wen [51] surveyed *timing attacks* on ML systems and laid conceptual groundwork,

but predates the NMS-overload and LLM-DoS literature. Most closely related is the SoK of Rathnasuriya et al. [88], which systematizes efficiency-robustness attacks on *dynamic* deep learning systems along three dynamic behaviors—per-iteration computation (D1), number of iterations (D2), and number of outputs (D3)—a taxonomy that parallels our treatment of adaptive networks, autoregressive generation, and object-detection output explosion; our survey extends beyond that dynamic-systems scope to the serving layer, embodied VLA policies, and agentic pipelines, and reorganizes the field around the intermediate-work amplification principle and the work-budget defense abstraction rather than around architectural dynamism. The two most closely related recent surveys each cover part of this landscape: Brachemi Meftah et al. [7] take a model-centric view of energy-latency attacks, and Zhang et al. [128] review resource-consumption threats specifically within the LLM pipeline, from threat induction through mechanism to mitigation. Our survey adds three things beyond these: it unifies latency attacks and defenses across perception pipelines, dynamic networks, and LLM/VLM serving under a single threat model; it additionally examines emerging embodied-AI and agentic attacks while distinguishing direct computational-latency evidence from adjacent denial-of-action and integrity threats; and it bridges the autonomous-driving perception literature that these surveys omit, organizing the whole around the intermediate-work amplification perspective. Table 1 represents this distinction, positioning our work against prior surveys along three coverage axes—autonomous-driving (AD) perception, general deep learning, and the LLM/VLM era—and identifying the principal gap each leaves open.

2.5 Metrics and a working formalization

Reported results in this field are often incomparable. Authors may report multiplicative latency increases ($\times$), absolute milliseconds, FLOP increases, energy in joules, generated-token counts, frames per second, and downstream task outcomes (crash rate, collision rate). On embedded targets, the relevant quantity is often the worst-case execution time (WCET) relative to a real-time deadline, which has motivated dedicated execution-time prediction frameworks for concurrent DNN workloads on edge accelerators [35]. Throughout, we preserve each paper’s native metric and flag where direct comparison is unsound. Establishing common axes is itself an open problem (Section 10).

2.5.1 Work and resource ratios. To make “intermediate-work amplification” precise and to separate it from ordinary large-input denial-of-service, we define a small set of quantities against a *matched benign reference set* rather than a single hand-selected reference input, whose choice could otherwise produce unstable or misleading ratios. Let $W(\cdot)$ be a stage-specific work measure (e.g., retained candidate boxes, activated neurons, retained tokens, selected experts, generated tokens, or decoder steps), and let $\mathcal{B}(x)$ denote a set of benign inputs matched to the adversarial input x . To make the match reproducible we recommend: *exact* matching of input dimensions, modality, model, decoding configuration, batch size, hardware, and concurrency; *stratified* matching of task category and benign input/output length; *optional* matching by a predeclared benign difficulty measure; and *no post hoc* matching using attacked latency or attacked output length (which would let the attack influence its own baseline). When an attack modifies a clean source input, output-length matching should use the clean source’s output or a predeclared benign stratum—never the attacked output—so the baseline cannot depend on the very effect the metric measures. Because a median over a small $\mathcal{B}(x)$ can still be unstable, evaluators should predeclare and justify the reference-set size and report bootstrap confidence intervals; larger sets are needed when the benign cost distribution is heavy-tailed or multimodal. Bootstrap at the independent experimental unit: matched input pairs for per-input attacks, complete workload windows (or block/run-level resampling) for time-correlated serving experiments, and complete runs or seeds where repeated measurements

Table 1. Positioning of this survey against prior surveys touching latency, timing, or availability attacks. ●: substantial coverage; ◐: partial/peripheral coverage; ○: not covered.

Survey	Year	AD percep.	General DL	LLM/VLM	Principal gap
Kianpour & Wen [51]	2019	○	◐	○	Predates modern latency/sponge and LLM-DoS literature
NIST AML taxonomy [104]	2023	○	◐	◐	Definitional only; availability is one small category
LLM-security surveys [1, 34, 62, 120]	2024–25	○	○	●	Serving-layer DoS only; no link to perception latency
VLM adversarial surveys [18, 27, 43]	2025–26	◐	○	◐	Jailbreak/backdoor/patch focus; omits output-length & resource attacks
Brachemi Meftah et al. [7]	2026	◐	●	◐	Focuses on model-level energy-latency attacks; does not synthesize recent serving, reasoning, agentic, and embodied systems
Rathnasuriya et al. (SoK) [88]	2025	◐	●	◐	Scoped to dynamic-DL efficiency robustness; omits serving, agentic, and embodied layers
Zhang et al. [128]	2026	○	◐	●	LLM resource-consumption pipeline only; no perception or embodied coverage
This survey	2026	●	●	●	— (unifies all three under a common availability threat model, with intermediate-work amplification explaining a major subset)

share system state. Because both the attack value and the benign reference are noisy, uncertainty should reflect both between-input variation and within-input system noise, and the estimand must be stated explicitly. For attacks derived from a clean source, report paired ratios between each attacked input and its clean counterpart. For attacks without a unique clean counterpart, divide by a predeclared statistic of a matched benign reference set and bootstrap the complete attack/reference-set construction; the latter is not a paired-ratio distribution and should not be labeled as one, because no pairing rule exists. In either case, repeated timing measurements of the *same* input characterize measurement variability and must not be treated as independent attack samples: independent

attack inputs characterize attack-population variability, and independent workload runs or seeds support serving-level inference. Throughout this survey we use *amplification factor* generically for the ratio between an attack-induced metric and a matched benign reference,

$$\text{AF}_M(x^{\text{adv}}) = \frac{M(x^{\text{adv}})}{\text{median}_{x_0 \in \mathcal{B}(x^{\text{adv}})} M(x_0)}, \quad (1)$$

where x^{adv} is the adversarial input, $M(\cdot)$ is the metric of interest—work (retained boxes, activated units, decoder steps, generated tokens), wall-clock latency, or energy—and $\mathcal{B}(x^{\text{adv}})$ is the predeclared matched benign set above. For source-derived attacks, the corresponding generic amplification factor uses the clean counterpart directly in the denominator rather than the median of a reference set. Depending on M , this quantity expresses work, latency, energy, token, or another form of amplification. Because published studies use heterogeneous definitions and experimental settings, we preserve each paper’s native metric rather than retrospectively converting all results to a common factor, and we introduce no separate named ratios beyond this generic form. One caution carries across metrics: a work-proxy increase need not produce a proportional latency increase, since wall-clock time can be bounded by memory movement, kernel-launch overhead, or batching rather than nominal work. When the benign reference median is *zero*—e.g., zero retained detections, zero selected optional components, or zero generated continuation tokens—the ratio is undefined and should not be rescued with an arbitrary ϵ ; report instead the *absolute* change together with the attack value, and for very small nonzero denominators report the denominator explicitly, since a large ratio may reflect a tiny baseline rather than a large operational effect.

Request-level latency alone is insufficient for shared serving systems, because attacks may primarily affect queueing delay, benign co-tenants, tail latency, or useful throughput. Evaluations should therefore report P50, P95, and P99 latency under benign and adversarial workloads, benign-victim latency under adversarial co-load, and the relative degradation in throughput or goodput (successfully completed useful requests per unit time) against a matched benign *workload*—a distinct experimental unit from a single input, since a workload also fixes an arrival process, request mixture, concurrency, batching, scheduling policy, and observation interval, all of which should be specified. This throughput or goodput degradation is what matters for scheduler, KV-cache, guardrail, and agentic attacks.

2.5.2 Deadline effects. On real-time targets, let p_{clean} and p_{attack} denote the fractions of frames missing their deadline under benign and adversarial input. We distinguish the *relative deadline-miss amplification* $p_{\text{attack}}/p_{\text{clean}}$, which is undefined when the clean miss rate is zero, from the *absolute deadline-miss increase* $p_{\text{attack}} - p_{\text{clean}}$, and recommend the absolute form as the default since well-engineered real-time systems have near-zero clean miss rates. To connect these ratios to the real-time-systems view, we also recommend reporting deadline slack and tardiness,

$$S_i = d_i - f_i, \quad \tau_i = \max(0, f_i - d_i) = \max(0, -S_i), \quad (2)$$

where d_i and f_i are job i ’s absolute deadline and completion time, S_i is slack, and τ_i is tardiness; when every job shares a common relative deadline D this reduces to $S_i = D - T_i$, and negative slack indicates a miss. Useful outcome measures include the deadline-miss rate, the distribution of slack, the maximum or high-quantile tardiness (stating whether the quantile is taken over frames, requests, or independent runs), the maximum number of consecutive missed deadlines in temporal systems, the recovery time after attack cessation, the backlog at attack termination, and—for streaming perception—the age of information where appropriate.

2.5.3 *Attacker economics.* The operational significance of any amplification factor depends on *attacker cost*. Evaluation should distinguish the attacker’s one-time development cost (e.g., optimizing a universal patch or trigger) from the marginal cost of each delivery attempt (e.g., issuing one API request or displaying the patch), and should count failed attempts when estimating campaign cost, since those still cost the adversary. Victim cost should measure the *incremental* resource consumption caused by the attack rather than total service cost, so that normal service cost is not misattributed to the adversary. Ratios between attacker and victim costs are meaningful only when both are converted to a common unit, such as monetary cost. For safety-critical attacks, attacker cost should be reported but should not by itself determine whether the threat is consequential: a safety-motivated attacker may willingly spend substantial resources to delay a single vehicle action. Reporting a cost–impact pair—an amplification factor *together with* the threat model and attacker effort—is, in our view, the single most useful discipline the field currently lacks; most surveyed papers report only the victim-side ratio. This framing also delimits our *unifying mechanism*: intermediate-work amplification occurs when an adversarial input whose externally visible size and request configuration are matched to a benign reference induces substantially greater *internal* intermediate work, as distinct from request flooding, oversized inputs, or naturally hard instances, all of which raise work without this matched-size asymmetry. The mechanism definition is not the survey’s complete inclusion criterion, however: scheduler- and agent-level attacks (Section 6) may fall outside the narrow matched-size asymmetry yet remain in scope as broader computational-availability threats.

3 Threat Models

A coherent threat model is the prerequisite for comparing attacks that span sensor inputs, public APIs, and hardware faults; without it, headline slowdown numbers are not commensurable. We classify latency attacks along three dimensions: the *attacker interface* (how the adversary touches the system), the *attacker knowledge* (white-, grey-, or black-box), and the *targeted stage*. Table 2 summarizes representative families.

Two distinctions are worth stating explicitly, because the rest of the survey preserves them. First, we separate the *security effect* from the *persistence mechanism*, since these are different dimensions rather than three values of one. *Availability degradation*—inflated computation, energy, latency, or missed deadlines—is the principal security effect studied here; *timing leakage*, in which data-dependent runtime *reveals* internal state rather than degrading service, is an adjacent *confidentiality* effect that shares the observable (variable runtime) but has the opposite goal, and we flag it as such wherever it arises rather than folding it into the amplification mechanism. Persistence is a separate property describing how long an attack remains effective: training-time poisoning and weight tampering (Section 7) are *persistent delivery mechanisms* that can embed availability degradation durably across subsequent inputs, rather than being a security consequence in their own right. Second, the attacker interfaces we group under one umbrella differ radically in access, cost, persistence, detectability, and feasibility: per-input adversarial examples, universal perturbations, physical patches or projector attacks, request-level API prompts, distributed request flooding, training-data poisoning, weight tampering, hardware fault injection, and timing side-channel exploitation are not interchangeable. Our synthesis abstracts over the *mechanism* (intermediate-work amplification) while retaining these interface distinctions in every comparison, since a projector attack and an API prompt that both achieve a 10× slowdown are not equally practical threats.

Table 2. Threat-model families for latency attacks, with representative works.

Family	Interface	Targeted stage	Representative evidence
Digital input perturbation	Modify sensor input (image, point cloud, text)	Detector inference / NMS / decoder	Overload [9]; Phantom Sponges [94]; sponge examples [95]
Pipeline-level perturbation	Modify camera input for end-to-end effect	Full perception (detect + track)	SlowTrack [72]
Physical projector / patch	Project or print patterns into the scene	Camera perception via phantom objects	SlowPerception [73]; DetStorm [78]
Sensor runtime maximization	Perturb LiDAR point clouds	LiDAR detection runtime	SlowLiDAR [64]
Cooperative V2V injection	Inject perturbations via V2V messages	Cooperative-perception fusion	CP-FREEZER [106]
Autoregressive inflation	Craft input prompt / image	Decoder generation length	NMTSloth [13]; Engorgio [20]; Verbose Images [29]
Dynamic-control defeat	Craft input to defeat conditional compute	Early-exit / pruning / routing	DeepSloth [42]; No-Skim [125]; Misrouter [24]
Training-time poisoning	Poison training data or weights	Persistent inference cost	Sponge poisoning [16]; Skip-Sponge [101]; DoS poisoning [30]
Hardware fault injection	Modify bits, parameters, or hardware state	NMS / weights / accelerator (persistent availability)	Bit-flip attack [96]; BitHydra [122]
Timing and serving side channels	Observe elapsed time, packet size, iteration boundaries, or token counts	Timing/serving oracle (confidentiality; adjacent, Section 7)	Timing side-channel [79]; speculative-decoding serving leak [99]

3.1 Attacker interface

The interface ranges from purely digital (modifying the input tensor of a model under white-box control) to physical (projecting or printing adversarial patterns that a camera then captures) to systemic (injecting messages over a vehicle-to-vehicle channel, or submitting prompts to a public API). For sensor-facing attacks, physical realizability is an important distinction between digital demonstrations and attacks deliverable through the sensing channel, and is a recurring theme in the autonomous-driving literature [73, 78]; it is not the dividing line for deployability in general, since API prompts, corpus poisoning, and remote serving attacks can be deployable without any physical realization.

3.2 Attacker knowledge

White-box attacks assume gradient access and dominate early work because the optimization is direct. Grey-box attacks assume architectural knowledge but not exact weights [21]. Black-box attacks, increasingly important for deployed services, rely on transfer from a surrogate, on query feedback (e.g., `usage.prompt_tokens`), or even on timing itself as an oracle [79, 130]. The existence

of timing-only black-box attacks is significant: it means even an opaque service that returns no probabilities can leak enough information to be optimized against.

4 Latency Attacks on Object Detection and Perception

Object detection is the most thoroughly studied target, because its post-processing stage offers an unusually clean computational bottleneck and because autonomous driving supplies a high-stakes deployment context.

4.1 Non-maximum suppression as a bottleneck

NMS filters overlapping candidate boxes and has a cost that grows with the number of candidates. *Phantom Sponges* [94] crafts a universal adversarial perturbation that adds many “phantom” boxes to overload NMS while *preserving* the original detections, so accuracy collapse does not betray the attack. *Overload* [9] generalizes this by formulating latency maximization as an optimization with a spatial-attention mechanism, reporting that single-image inference time can be increased tenfold; the authors describe the attack as “NMS-agnostic”—meaning the optimization is not tied to one particular NMS implementation, not that the attack is independent of candidate-dependent detector processing, since it still operates by proliferating candidate detections. *Beyond PhantomSponges* [93] modifies a box-area loss to lower intersection-over-union and exacerbate NMS load, reporting up to a 550% increase in NMS time on a YOLOv5-small configuration. *Groundswell* [116] sharpens the stealth-effectiveness trade-off with a *Regional Perturbation Balance* strategy that injects phantom objects into spatial regions that minimally disturb the original detections; on YOLOv5s it inflates the candidate count fed to NMS from 101 to roughly 19,000 and end-to-end latency from 15.2 ms to 41.4 ms (about a 172% increase), while cutting optimization time by 80% relative to prior universal-perturbation attacks. *Daedalus* [107], although framed as an integrity attack, is a relevant NMS failure mode: it compresses box dimensions so NMS cannot filter dense false positives, and is demonstrated via printed posters. A complementary observation by Biton et al. [5] reframes variable-time inference as a security liability in itself: because NMS runtime is data-dependent, it *leaks* internal inference state, and this timing signal can be used to enhance black-box, decision-based attacks. This dual role—NMS as both a target of latency inflation and a side-channel oracle—motivates constant-time NMS as an explicit security requirement (Section 8).

4.2 Whole-pipeline and temporal-state attacks

A typical autonomous-driving perception stack feeds detector outputs into a multi-object tracker (MOT) that maintains a persistent identity for each object across frames. Many real-time trackers follow the *tracking-by-detection* paradigm: every frame, the tracker (i) predicts each existing track’s new position—often with a per-track *Kalman filter* using a constant-velocity motion model, as in SORT-family pipelines—(ii) *associates* the incoming detections to those predicted tracks, and (iii) updates matched tracks, spawns tracks for unmatched detections, and retires tracks unseen for a buffer of frames. The association step—the stage that tracker-overload attacks target—is commonly solved as a bipartite matching (Hungarian or a related assignment method) over a cost matrix built from every (track, detection) pair, though this is not universal: some trackers instead use greedy association, learned association, joint detection-and-tracking, or alternative motion models. For M active tracks and N detections, constructing the dense association matrix costs $O(MN)$; the assignment step’s complexity depends on the algorithm and how rectangular matrices are handled, with a common coarse upper bound of $O(\max(M, N)^3)$. Increasing N therefore raises current-frame association work directly, while persistent false tracks can increase future association-matrix dimensions, causing cumulative workload growth across frames; the exact scaling depends on the track-retirement policy and the assignment implementation. Unlike a fixed-structure detector

forward pass, such a tracker is *content-dependent*: flooding it with detections directly increases its per-frame workload.

Detector-only attacks often fail to change downstream behavior because multi-object tracking absorbs detection noise; prior work shows that over 98% detection-attack success may be required to alter tracking outcomes [48]. This motivates pipeline-level threat models. *SlowTrack* [72] attacks camera-based perception end to end with a two-stage strategy and three new loss designs, reporting average latency $2.9\times$ higher than prior approaches for the whole camera-based AD perception system; under its evaluated Baidu Apollo stack and LGSVL simulator configuration, the authors further report a vehicle crash rate of $\sim 95\%$ versus $\sim 30\%$ for baselines. This links induced latency to a downstream safety outcome for that specific stack and simulator setup rather than in general. Related temporal-integrity work—tracker hijacking [48, 71] and trajectory-aware attacks on MOT [44]—reinforces that stateful components can amplify or attenuate component-level perturbations.

4.3 Physical-world latency attacks

SlowPerception [73] introduces projector-based universal perturbations that create numerous phantom objects; under the authors' specified industry-grade system and simulator configuration, they report an average physical-world latency of 2.5 seconds and a 97% collision rate. As with all such downstream-outcome figures, this collision rate is specific to the evaluated vehicle stack, speed, control policy, perception frequency, and fallback behavior, and should not be read as a configuration-independent consequence of detector slowdown. *DetStorm* [78] advances physical realizability, optimizing objects against multiple NMS variants and reporting a 506% average increase in detected objects and delays up to 8.1 seconds, while arguing that prior digital or large-patch attacks lack real-world applicability. *Groundswell* [116] is explicitly designed for physical delivery: the universal perturbation can be affixed as a sticker on the camera lens or projected onto a surface ahead of the vehicle, injecting itself into raw frames before in-vehicle processing and pushing per-frame latency past the source paper's 30 ms real-time budget adopted for its evaluated SAE Level 4–5 automation configuration (SAE levels do not define a single universal perception deadline; acceptable latency depends on speed, sensor rate, braking distance, architecture, and control policy).

4.4 Beyond cameras: LiDAR and cooperative perception

SlowLiDAR [64] presents what its authors describe as the first systematic availability attack on LiDAR detection, handling non-differentiable preprocessing via differentiable proxies and an execution-time-aware loss to inflate runtime across six popular pipelines while maintaining imperceptibility. *CP-FREEZER* [106] targets vehicular cooperative perception, injecting adversarial perturbations via vehicle-to-vehicle (V2V) messages to maximize fusion delay; it resolves point-cloud nondifferentiability and transmission asynchrony, increasing end-to-end latency by over $90\times$, pushing per-frame time beyond 3 seconds, and achieving 100% success on a real vehicle testbed. A simulation study further analyzes the system-level impact of perception inference-time attacks on full autonomous vehicle stacks [10], and real-time defenses against object-based LiDAR attacks are beginning to appear [127].

4.5 Embodied and vision-language-action availability attacks

Work appearing in 2025–2026 moves past the perception front-end to the *Vision-Language-Action* (VLA) models that map multimodal observations directly to robot actions, where an availability failure is a physical failure: a robot that stalls or acts late can collide, drop a payload, or freeze mid-manipulation. We emphasize at the outset that most attacks in this space are *availability-adjacent* rather than latency attacks under the definition of Section 2.5: they deny, halt, or corrupt action

Table 3. Representative latency attacks on object detection and perception. Metrics are reported as in the source; “–” indicates not reported in a directly comparable form.

Method	Target stage	Key quantitative result	Delivery	Evaluation
Phantom Sponges [94]	NMS (end-to-end OD)	Universal perturbation overloads NMS while preserving detections	Digital	Digital model eval.
Overload [9]	OD inference (edge)	Single-image inference time up to 10x; NMS-agnostic	Digital	Target hardware
Beyond Phantom-Sponges [93]	NMS runtime (YOLO)	Up to 550% NMS-time increase (YOLOv5-small)	Digital	Digital model eval.
Groundswell [116]	NMS (camera OD)	Candidates 101→19k; 15.2→41.4 ms (~172%); –80% opt. time	Physical (lens sticker / projector)	Target hardware
SlowTrack [72]	Camera perception (detect+track)	Latency 2.9x vs. prior; crash rate ~95% vs. ~30%	Digital	Closed-loop simulation
SlowPerception [73]	NMS + MOT workload	Avg. 2.5 s physical latency; 97% collision rate	Physical (projector)	Closed-loop simulation
DetStorm [78]	Camera perception delay	506% more detections; up to 8.1 s delay	Physical (projector)	Target hardware
SlowLiDAR [64]	LiDAR detection runtime	Significant latency rise on six LiDAR pipelines	Digital	Digital model eval.
CP-FREEZER [106]	Cooperative fusion	> 90x latency; >3 s/frame; 100% on testbed	V2V message	Vehicle testbed
Sponge Backdoor [117]	OD latency via NMS	Poisoned inputs prolong time; clean accuracy preserved	Training poison	Digital model eval.
Bit-flip attack [96]	NMS via parameters	Up to 71.6 ms (20.4x) with 31 bit-flips	Hardware fault	Target hardware

Notes. Figures use each source’s native measure (candidate-box count, wall-clock latency, FPS, or a downstream outcome such as collision/crash rate) and are *not* directly comparable across rows; each ratio is relative to that paper’s own benign baseline. Downstream-safety figures are specific to the evaluated vehicle stack, speed, control policy, and simulator configuration. We separate *Delivery* (how the attack reaches the system: *Digital* input, *Physical* sensor delivery with the specific channel noted, *V2V message*, *Training poison*, or *Hardware fault*) from *Evaluation* (where effects are measured: *Digital model evaluation* = tensor-level experiment on the trained model, *Target hardware* = timed on the deployment device, *Closed-loop simulation* = simulator with control outcomes, *Vehicle testbed* = closed physical system). Where an entry lists both a physical delivery and a target-hardware measurement, these were reported by the source but not necessarily in a single combined on-road experiment; no entry denotes a full on-road deployment.

without demonstrating computational-work or latency amplification. *FreezeVLA* [110] formalizes *action-freezing* attacks via min-max bi-level optimization: a single adversarial image “freezes” the VLA and makes it ignore subsequent instructions, attaining a 76.2% average freeze rate across three VLA models and four benchmarks with strong cross-prompt transferability; the reported outcome is denial of action, not a measured increase in computation. *Semantic-DoS* [98] turns the model’s own safety alignment into the attack surface: injecting short (1–5 token) safety-plausible phrases into the robot’s audio channel triggers protective halting without any jailbreak, reaching up to 98.3% hard-stop success—service denial through triggered halting, again without intermediate-work amplification. A third group is best read as *integrity attacks with hypothesized latency implications*: the Embedding Disruption Patch Attack (*EDPA*) [119] places a model-agnostic patch in the camera view that breaks visual–textual alignment and drives repeated incorrect actions to task failure,

and MAVLA [131] breaks cross-modal alignment through a coordinated multimodal adversarial framework—in both cases the demonstrated outcome is incorrect or failed action rather than measured slowdown. ANNIE [47] provides a systematic, ISO-grounded study of embodied *safety* violations, decomposing long-horizon goals into frame-level perturbations and exceeding 50% success across critical, dangerous, and risky categories with validation on a physical robot.

These embodied attacks demonstrate that delayed or absent actions can create physical safety consequences. However, not all are computational latency attacks under our definition. Freeze or protective-halt attacks deny progress without necessarily increasing intermediate work, while cross-modal disruption attacks primarily target integrity. We therefore treat them as availability-adjacent evidence and reserve the latency-attack category for studies that explicitly measure or optimize computation, response time, energy, or missed deadlines. Within this section, only the reflection-backdoor attack on driving VLMs [65] (Section 6.2) meets that bar for embodied platforms, and the appendix catalog labels each embodied entry accordingly. Broader treatments of the embodied threat surface are emerging in dedicated surveys [118].

Section Summary. Collectively, these studies illustrate a clear evolution of latency attacks in perception systems. Early work primarily exploited a single computational bottleneck, most notably non-maximum suppression (NMS). More recent research increasingly targets complete perception pipelines, incorporates physical realizability, and evaluates downstream consequences such as tracking failures, delayed planning, and collision risk, while recent embodied-VLA work illustrates *adjacent* denial-of-action and integrity threats rather than demonstrated computational-latency amplification (Section 4.5). This progression marks a step toward system-level thinking: latency attacks are no longer isolated component failures but increasingly propagate across perception pipelines. More importantly, the object-detection literature reveals that the attacker’s objective is not to exploit NMS itself, but to maximize intermediate computation wherever it occurs. The following sections show that this principle naturally extends beyond perception to general neural inference.

5 Neural Inference Work-Inflation Attacks

The *sponge* lineage targets the general mechanism of activation sparsity rather than any specific post-processing stage, making it broadly applicable across architectures.

5.1 Inference-time sponge examples

Sponge examples [95] are the seminal work: adversarial inputs crafted to maximize energy and latency by minimizing the activation sparsity that accelerators exploit. On transformer and vision models, they report up to 30× increases in latency and energy consumption in some settings, and a real-world demonstration that raised a cloud translator’s response time from 1 ms to 6 s. A follow-up analysis shows that uniform/constant inputs act as “free” sponge examples and that the achievable energy increase is bounded by the architecture’s sparsity range, motivating activation-sparsity monitoring as a defense signal [77].

5.2 Sponge poisoning: moving the attack to training

Sponge poisoning [16] injects a small fraction of crafted samples so the trained model incurs elevated energy on *all* natural inputs thereafter—a far stealthier and more persistent variant. *SkipSponge* [101] operates directly on weights, poisoning residual (skip) connections with under 1% weight modification and evading fine-pruning and parameter-perturbation defenses. Sponge poisoning has been adapted to on-device DNNs [115], to multi-exit networks (where it both prevents early exits and raises per-layer cost) [45], to mobile apps [84], and to FPGA accelerators under

differential privacy [2]. A sensing-AI study quantifies battery-drain risk in IoT settings and proposes pruning as a defense, reducing energy overhead from $\sim 4\times$ to $\sim 1.3\times$ [41].

5.3 Sparsity and spiking networks

Sparsity attacks [53] degrade energy/latency by minimizing activation sparsity and withstand activation thresholding and input quantization. Spiking neural networks, attractive for low-power edge perception, are also vulnerable: efficiency attacks raise spike activity by $1.7\times$ – $2.5\times$ and energy by $1.4\times$ – $2.2\times$ [54], and timestep-compressed methods reduce the latency of *generating* such attacks for real-time use [50].

5.4 Dynamic and adaptive networks

Input-adaptive networks promise lower *average* cost; latency attacks revoke that promise by forcing worst-case control flow.

Multi-exit networks. The earliest availability attack on input-adaptive networks is *ILFO* [37], which optimizes an intermediate-output loss to reactivate skipped blocks in depth-adaptive networks, increasing FLOPs by up to 100%. Building on ILFO, *GradAuto* [83] attacks both dynamic-depth and dynamic-width networks by carefully steering the direction and magnitude of the gradient to find near-imperceptible perturbations that reactivate skipped computation, recovering on average 100% of the FLOPs saved by the dynamic architecture; the related *GradMDM* [82] further refines gradient direction and magnitude to minimize perturbation perceptibility while maximizing activated computation. *DeepSloth* [42] is the foundational slowdown attack on early-exit inference: per-input perturbations force the network to defer to its final, most expensive classifier, eliminating the efficiency advantage with 90–100% success and up to $5.2\times$ cost inflation. Refined energy attacks combine energy maximization with confidence minimization and remain effective in grey-box settings [21]. The same principle extends to continuous-depth architectures: *AntiNODE* [38] formulates the input–latency relationship for neural ODEs and crafts inputs that inflate the number of adaptive solver steps, degrading efficiency by up to 335% under the authors’ metric (a relative increase in solver cost; a conventional relative efficiency reduction cannot exceed 100%) with adversarial examples that transfer across ODE solvers and architectures in a black-box setting—confirming that any architecture whose runtime is governed by a data-dependent stopping criterion is exposed. Whereas most of these attacks assume white-box gradients, *EREBA* [40] is presented by its authors as the first *black-box* energy-testing method for adaptive networks: it learns a proxy energy estimator and uses it to synthesize inputs that trigger worst-case computation without access to model internals, increasing energy consumption by up to 2,000% relative to the benign baseline and establishing that the energy-robustness gap is exploitable even for opaque, query-only deployments. On resource-constrained edge deployments, *DDAS* [3] makes the target of the attack explicit at the system level: rather than degrading accuracy, it forces edge dynamic neural networks onto their longest execution path specifically to drain IoT-device resources—battery, latency, and memory—demonstrating that the operative harm is resource exhaustion on the device itself rather than a wrong prediction. In NLP, *SlowBERT* [124] attacks multi-exit BERT, *Dynamic Transformers Provide a False Sense of Efficiency* [15] introduces a unified attack (SAME) that defeats early exits across BERT, RoBERTa, ALBERT, and DistilBERT with up to $5\times$ slowdown, and *WAFFLE* [17] systematically audits the slowdown robustness of three multi-exit language-model mechanisms in both white- and black-box settings, additionally finding universal slowdown triggers and showing that adversarial training fails while input sanitization (e.g., via a conversational model) is comparatively effective. Beyond attack crafting, *DeepPerform* [12] recasts the problem as software *performance testing*: it trains a generator to efficiently synthesize test inputs that approximate a dynamic model’s worst-case latency and energy, enabling systematic pre-deployment auditing of

efficiency degradation. A foundational study demystifies *why* these models are vulnerable and proposes calibration-based defenses that recover roughly 60% of lost efficiency [103].

Token pruning, skimming, and mixture-of-experts. Skimming language models drop unimportant tokens; *No-Skim* [125] crafts inputs that prevent dropping, raising FLOPs by 2–3×. A unified framework treats early exit, token-pruning vision transformers, and MoE routing as instances of one “efficiency attack” class, demonstrating 2–8× overhead across all three [89]. In the vision-transformer setting specifically, *SlowFormer* [80] trains a universal adversarial patch (as little as 2% of the image area) that drives efficient ViTs such as A-ViT, ATS, and AdaViT toward their maximum compute; *DeSparsify* [123] similarly targets the availability of token-sparsification mechanisms, crafting stealthy perturbations that force ATS/AdaViT/A-ViT to retain all tokens (raising GFLOPs by 44–100%) while preserving the clean prediction; and *QuantAttack* [4] exploits dynamic quantization to force worst-case high-precision execution, inflating inference time, memory, and energy. Mixture-of-experts routers, central to modern sparse LLMs, have also been targeted: *Misrouter* [24] causes routing collapse or saturation in a black-box, query-only setting, yielding 2–4× latency increases, while *RouteHijack* [121]—though primarily a jailbreak attack—demonstrates that adversarial control of expert routing can additionally inflate computation by activating costly experts. *RepetitionCurse* [46] shows the vulnerability is systemic rather than incidental: under expert parallelism, simple repetitive-token prompts force all tokens onto the same top- k experts, creating device-level bottlenecks while others idle and raising end-to-end latency by 3.06× on Mixtral-8x7B—turning the efficiency mechanism itself into a denial-of-service vector. *SPLAT* [58] revisits latency attacks on dynamic networks with a black-box, stealthy formulation for multi-exit models.

Section Summary. Unlike perception attacks, sponge attacks demonstrate that latency vulnerabilities are not tied to a particular application or post-processing stage. Instead, they arise from architectural properties shared by many neural networks, including activation sparsity, conditional computation, and dynamic execution. Training-time variants further show that excessive computational cost can be embedded persistently into model parameters rather than triggered solely by adversarial inputs. Together, these studies shift the focus from application-specific bottlenecks toward the computational mechanisms that underlie efficient neural inference.

6 Autoregressive and Serving-Side Work Inflation

6.1 Sequence generation and decoder-length inflation

Autoregressive decoders give attackers a direct lever over the *number of generated tokens*. Total cost generally increases with that count, while per-token cost may also vary with context length, KV-cache behavior, batching, and the attention implementation (consistent with the complexity discussion in Section 2.3).

NMTSloth [13] is presented by its authors as the first systematic study of efficiency robustness in neural machine translation, crafting source sentences that inflate decoder steps by 5–10× and end-to-end latency by 3–8× while leaving BLEU nearly unchanged, so a monitor watching only BLEU may not flag the attack. *LLMEffiChecker* [25] extends this to general LLMs, finding them *more* vulnerable: a five-token perturbation can inflate generation length by 10–30×. At the input-encoding level, *Bad Characters* [6] shows that imperceptible Unicode perturbations—invisible characters, homoglyphs, reorderings, and deletions—can be injected into text in a black-box setting; while primarily an integrity attack, the same encoding-level manipulation could plausibly provide a stealthy delivery channel for the efficiency-degradation objectives above against machine-translation and other text pipelines. *NICGSlowDown* [14] brings the paradigm to neural image captioning, forcing decoders to emit far longer captions via image perturbations, and contributes an efficiency-robustness

benchmark reused by later work. *TTSlow* [31] demonstrates that the threat reaches audio synthesis: adversarial text inflates autoregressive text-to-speech decoding by up to 5×. The complementary direction, audio *recognition*, is covered by *SlothSpeech* [39], presented by its authors as the first efficiency-robustness attack on automatic speech recognition: small perturbations to the input waveform prolong the decoder’s output, inflating latency to up to 40× baseline and showing that both ends of the speech pipeline—synthesis and recognition—share the autoregressive availability vulnerability. A property common to these attacks, and a recurring defensive blind spot, is that each preserves its source paper’s reported task metric (e.g., BLEU or caption quality), so monitors based only on that metric may fail to detect them—though extremely long or repetitive outputs can still degrade practical utility.

6.2 LLM and VLM resource-consumption attacks

The LLM/VLM era inherits the autoregressive lever and adds new ones: chain-of-thought reasoning, multimodal inputs, and shared serving infrastructure. Reasoning models substantially enlarge an existing attack surface: input-dependent computation was already present in NMS, adaptive ODE solvers, early exits, tracking state, and autoregressive translation, but reasoning models expose a far wider controllable range of sequential inference work through variable reasoning length.

Output-length inflation and inference-cost DoS. *Engorgio* [20] optimizes prompts that coerce LLMs into prolonged low-information generation, inflating output token counts by 10–50× and transferring to black-box targets. *Crabs* [130] automates black-box LLM-DoS prompt generation, sustaining 10–30× output inflation even under provider-side length monitoring through semantic camouflage. A recurring weakness of these EOS-delay methods is that control over the termination symbol erodes as the output grows; *LoopLLM* [57] circumvents this by inducing *repetitive generation*—a repetition-inducing prompt drives the model into a low-entropy decoding loop that reliably runs to the maximum output length (over 90% of the cap versus ~20% for EOS-delay baselines), and a token-aligned ensemble optimization improves cross-model transferability by roughly 40% to commercial models such as DeepSeek-V3 and Gemini 2.5 Flash. *Denial-of-Service Poisoning* [30] shows the threat can be embedded at training time: a triggered model emits endless output, never producing EOS, with 100% success at under 1% poisoning and survival through RLHF safety tuning. The attack surface also expands with the surrounding pipeline: for retrieval-augmented generation, *RA-ICA* [63] poisons the external knowledge corpus rather than the prompt—injecting documents that are semantically retrievable yet induce abnormally long generations—raising token consumption by up to 13.1× with over 90% success while leaving answer integrity intact, a threat model that is a distinct and potentially practical delivery model in systems that ingest or retrieve attacker-publishable content, though its effectiveness depends on controlling retrieval rank, corpus ingestion, and trust filters. *DrainCode* [114] specializes this corpus-poisoning vector to retrieval-augmented code generation: gradient-guided mutation crafts syntactically valid but semantically inert triggers that suppress early EOS while a KL-divergence constraint preserves functional correctness, inflating output length 3–10×, latency by 85%, and energy by 49% while retaining 95–99% pass rate and evading both classifier- and perplexity-based filters—showing that a code-assistant RAG corpus can serve as a stealthy token-exhaustion channel.

Reasoning-model and “infinite-thinking” attacks. Inference-time compute scaling creates a new, high-leverage surface. *ReasoningBomb* [66] induces pathologically long reasoning traces in large reasoning models—averaging ~18,759 completion tokens versus ~300 for normal queries, a roughly 60× cost multiplier—while remaining stealthy and bypassing input-, output-, and joint-detection baselines at rates of 99.8%, 98.7%, and 98.4%, respectively. *ThinkTrap* [59] exploits chain-of-thought mode in black-box APIs to induce near-infinite thinking loops, consuming 50–100× more tokens than standard requests. *OverThink* [55] makes the delivery indirect, which can be more

practical in deployments that retrieve external content: it injects *decoy* reasoning problems into public content that the model later retrieves (e.g., a web page or a RAG source), amplifying reasoning tokens by up to $46\times$ while keeping the final answer contextually correct, so the victim need never receive a malicious prompt directly. *ExtendAttack* [135] instead hides the trigger inside benign queries by obfuscating characters into a poly-base ASCII encoding, compelling computationally intensive decoding sub-tasks that lengthen responses by over $2.5\times$ on o3 while preserving meaning and accuracy. Because these attacks preserve the reported answer quality and several bypass detection at high rates, they motivate budget enforcement by *construction* rather than detection (Section 8).

Multimodal verbose attacks. *Verbose Images* [29] crafts adversarial images that make large VLMs (GPT-4V, LLaVA, InstructBLIP, MiniGPT-4) generate long, repetitive responses, inflating output by $10\text{--}30\times$ and latency by $7\text{--}30\times$ while preserving visual fidelity. *VLMInferSlow* [112] provides a service-oriented evaluation across 12 open and closed VLMs, reporting 85–100% success and $5\text{--}40\times$ increases in latency. *Verbose-Text Induction* [70] co-optimizes adversarial images and prompts to exceed 10,000 output tokens from a single query, and *LingoLoop* [26] traps multimodal models in self-referential generation loops that fill the context window faster than legitimate output, exhausting the configured context-length cap (the cap is reached, not bypassed). These attacks extend the output-inflation lever from unimodal captioning [14] to image- and prompt-conditioned VLMs. The same lever also reaches the temporal domain: *VidDoS* [100] is described by its authors as the first universal energy-latency attack tailored for video-based LLMs, using a single instance-agnostic adversarial patch—trained via masked teacher forcing with a refusal penalty and early-termination suppression—that requires no inference-time gradients and therefore keeps pace with continuous video streams. It induces token expansion exceeding $205\times$ and latency inflation exceeding $15\times$ across three Video-LLMs; in the authors’ simulated real-time autonomous-driving streaming setup, the induced latency is reported to cause safety violations, though this outcome is again specific to their simulation configuration. The threat has also reached *embodied* multimodal autonomy: a natural-reflection backdoor attack on driving VLMs [65] embeds reflection-pattern triggers in camera images that, when present, cause the VLM planner to generate abnormally long responses; the demonstrated latency effect is *prolonged VLM generation* (increased output-token count and correspondingly higher inference latency under the trigger), which is a measured computational-cost increase rather than a mere task-denial response delay, uniting the output-length-inflation and training-time-backdoor threat classes in a single AD-relevant attack.

Serving-framework, tool-chain, and agentic DoS. A further set of attacks moves below the model to the *serving system* and the *agentic loop*, where shared infrastructure amplifies a single adversary’s leverage. *Fill and Squeeze* [108] reports that, under its evaluated continuous-batching configurations, length inflation alone had less impact than direct scheduler- and KV-cache-targeted pressure, and accordingly attacks the scheduler directly—exhausting the global KV cache to induce head-of-line blocking and forcing repetitive preemption—for up to $20\text{--}280\times$ time-to-first-token slowdown in a black-box setting at lower cost than input-only methods. At the agent layer, *Beyond Max Tokens* [132] exploits the Model Context Protocol tool interface: text-only edits to tool metadata, optimized by Monte-Carlo tree search, steer agents into prolonged tool-calling chains exceeding 60,000 tokens, raising per-query cost by up to $658\times$ while preserving task success and evading prompt filters. *From Shield to Target* [133] turns LLM *guardrails* into the victim: crafted payloads trap the guardrail in extended reasoning loops for $13\text{--}63\times$ token amplification and up to $148\times$ latency amplification, and a single poisoned document can starve co-located agents sharing the guardrail. *Mobius Injection* [61] generalizes this to agent-based DDoS (ABo-DDoS): by exploiting a “semantic closure” vulnerability, one textual injection induces sustained recursive agent execution, driving single-node call amplification up to $51\times$ and multi-node p95 latency up to $229\times$. These

developments echo the broader systematization of the agentic attack surface [19] and are now recognized at the standards level as “unbounded consumption” [81]—reinforcing the need to treat availability, not just integrity, as a first-class security property of deployed LLM services.

Section Summary. Earlier attacks were constrained by the computational graph of a single model, whereas modern sequence models expose output length—and increasingly reasoning length—as attacker-controllable variables. As a result, computation itself becomes part of the attack surface. Recent work further demonstrates that availability attacks are no longer confined to individual models. The attack surface is rapidly expanding toward shared serving infrastructure, tool chains, and agent orchestration, where computational amplification compounds across multiple services rather than within a single forward pass. This progression marks a broader shift from model-centric latency attacks toward system-level availability threats.

7 Training-Time Persistence, Hardware, and Timing Channels

Not all latency attacks act on the test-time input. The attacks discussed in this section broaden the threat model beyond the test-time input: they persist across sessions, operate below the algorithm, or turn latency itself into an information leak, and are therefore the hardest to detect and remove. We include timing side channels here only to explain the dual security role of input-dependent runtime; they are a confidentiality phenomenon and are *not* counted as latency attacks in the survey’s availability taxonomy or in any quantitative attack totals. *Training-time* variants (sponge poisoning [16], SkipSponge [101], sponge backdoors in detectors [117], and no-EOS DoS poisoning of LLMs [30]) introduce persistent cost inflation that persists through normal usage and is hard to remove without retraining. *Hardware* attacks operate below the algorithm: a Rowhammer-based fault-injection attack flips selected parameter bits affecting NMS behavior, reporting up to 71.6 ms (20.4×) latency with just 31 bit flips [96], and FPGA sponge poisoning evades differential-privacy noise to inflate accelerator energy [2]. *BitHydra* [122] carries the bit-flip vector into the LLM era: formulating the search for cost-maximizing weight bits as a constrained binary integer program solved via ADMM, it suppresses the end-of-sequence probability so that generation continues until an external token, context, or execution limit is reached, with as few as 1–4 bit flips across ten LLMs (1.5B–16B), decoupling a persistent output-length DoS from any specific input query. *Side-channel* attacks invert the relationship between timing and computation: a timing-based black-box method crafts adversarial examples using only processing time, exploiting the link between activated nodes and latency, and evades gradient-masking countermeasures [79]. Speculative decoding introduces a *serving* side channel that is broader than a pure timing channel: the same duality now appears in modern serving optimizations: because speculative decoding accepts a data-dependent number of draft tokens per iteration, packet sizes and iteration boundaries can reveal the input-dependent number of accepted draft tokens, enabling query fingerprinting with over 90% accuracy and datastore extraction across production vLLM deployments [99]; because the observable is packet-size and iteration-boundary leakage rather than elapsed time alone, this is a serving/traffic-analysis side channel in which timing may participate but is not the sole channel. This is the mirror image of a latency attack—rather than the adversary inflating the victim’s computation, the victim’s variable computation inflates what the adversary can observe—and it shows that an optimization introducing input-dependent timing can open both an availability lever and a confidentiality leak. We flag this explicitly as a timing-*leakage* result (distinct from availability degradation; Section 3): latency here is not a target but an oracle.

Section Summary. These attacks demonstrate that latency is not solely an inference-time phenomenon. Computational overhead can be introduced during training, manipulated below the algorithmic level through hardware faults, or even exploited as an information leak through timing side channels. Together, these studies broaden the threat model beyond adversarial inputs and

Table 4. Representative latency/energy attacks beyond object detection. Native metrics preserved.

Method	Domain / target	Key quantitative result
Sponge Examples [95]	General DNN (vision, NLP)	Up to 30× energy/latency; cloud translator 1 ms→6 s
Sponge Poisoning [16]	Training-time, general	1–5% poison ⇒ persistent 2–10× energy
SkipSponge [101]	Weight poisoning	High energy overhead at <1% weight change; evades defenses
DeepSloth [42]	Multi-exit networks	90–100% force full inference; up to 5.2× cost
GradAuto [83]	Depth- & width-dynamic AdNNs	Recovers ~100% of reduced FLOPs
AntiNODE [38]	Neural ODEs	Source-reported equivalent of a 335% efficiency degradation (solver-cost metric; not a conventional bounded percentage); solver-transferable (black-box)
EREBA [40]	Adaptive networks (black-box)	Up to a 2,000% energy increase (≈21× baseline) via learned proxy estimator
Dynamic Transformers (SAME) [15]	Multi-exit NLP	Up to 5× slowdown across BERT family
No-Skim [125]	Skimming LMs	2–3× FLOPs; output label unchanged
Misrouter [24]	MoE routing (black-box, input-only)	2–4× latency via routing collapse/saturation
RepetitionCurse [46]	MoE expert parallelism	3.06× latency on Mixtral-8x7B via router imbalance
NMTSloth [13]	Neural machine translation	5–10× decoder steps; 3–8× latency; BLEU stable
NICGSlowDown [14]	Image captioning	5–10× decoding steps; 3–7× latency
Engorgio [20]	LLM prompts	10–50× output tokens; white-box, transfers to black-box
Crabs [130]	LLM-DoS (black-box)	10–30× output; evades length monitoring
ReasoningBomb [66]	Reasoning models	~60× tokens
Verbose Images [29]	VLMs	10–30× output; 7–30× latency
VLMInferSlow [112]	VLM-as-a-service	85–100% success; 5–40× latency
Fill and Squeeze [108]	LLM serving scheduler	20–280× TTFT slowdown; black-box
Beyond Max Tokens [132]	LLM agent tool chains (MCP)	Up to 658× per-query cost; >60k-token chains
From Shield to Target [133]	LLM guardrails	13–63× tokens; up to 148× latency
Mobius Injection [61]	Agentic AbO-DDoS	51× call amplification; 229× p95 latency

Notes. “What is measured” varies by row—generated-token count, FLOPs, wall-clock or time-to-first-token (TTFT) latency, or energy—so the multiplicative factors are *not* commensurable and each is relative to the source’s own benign reference.

Most entries are measured white- or black-box results; where an entry reports only a per-message or per-query amplification, victim-side cost is stated without a matched attacker-cost term (Section 2.5). Reported “success” retains each source’s own definition (e.g., reaching a length threshold, exceeding a latency threshold, causing a deadline miss, or inducing a selected route); thresholds differ across studies and are not directly comparable. Multiplicative factors (×) denote amplification relative to the benign baseline, whereas standalone percentages that name an attained outcome (e.g., a success or collision rate) are final values, not relative increases.

emphasize that securing availability requires reasoning about the entire deployment stack rather than the inference algorithm alone.

8 Defenses

Defenses are nascent, often attack-family-specific, and pervaded by an efficiency–robustness trade-off. We organize them by *control mechanism*—the point in the compute pathway each one actually bounds—rather than by the attack they were designed to counter. Table 6 summarizes the landscape.

8.1 Budget enforcement and compute caps

The most direct defense against autoregressive inflation and reasoning DoS is to enforce a budget by construction. *PD3F* [129] is a two-stage, pluggable framework that combines input-side dynamic request scheduling (via a “Resource Index”) with output-side “Adaptive End-Based Suppression”, which terminates excessive generation early, improving access capacity by up to 500% under adversarial load across six models. *Token-budget-aware pool routing* [11] estimates each request’s token budget and routes it to a right-sized pool, avoiding worst-case provisioning that wastes 4–8× concurrency. *Conformal Thinking* [111] reframes reasoning-budget allocation as a risk-control problem, stopping under a statistical criterion when the model is sufficiently confident or an instance appears unlikely to benefit from further computation. A complementary line of work, originally aimed at cost efficiency, may reduce exposure to reasoning-inflation attacks by encouraging or learning shorter chains of thought. At the *prompt* level, *token-budget-aware reasoning* (TALE) [36] injects a difficulty-estimated token budget into the prompt and dynamically adjusts the reasoning length per query, and *Concise CoT* (CCoT) [91] simply instructs the model to reason concisely, cutting response length by roughly 48% with negligible accuracy loss on non-mathematical tasks. At the *parameter* level, *CoT-Valve* [74] identifies a single tunable direction in weight space (via a lightweight LoRA branch) that elastically controls reasoning-chain length, compressing GSM8K chains from 741 to 225 tokens with a <0.2% accuracy drop—a model-native cap that does not rely on a possibly-attacker-controlled prompt. These mechanisms differ in the *strength* of the guarantee they provide, and conflating them would overstate the state of the art. Prompt-based methods such as TALE and Concise CoT *encourage* shorter traces, but a prompt instruction is not a cap: the model may exceed the requested length, and a malicious client can remove or alter the instruction. Parameter-level methods such as CoT-Valve *shift* the model toward shorter reasoning, but a learned length-control direction reduces observed length without certifying a maximum token count under adaptive attack. Runtime termination and token accounting, by contrast, *enforce* hard limits. Only the last category directly guarantees a configured upper bound; the others may reduce expected cost but require adversarial evaluation. Table 5 summarizes this spectrum. Across all strengths a further caveat applies: a fixed budget imposed *a priori* risks truncating genuinely hard reasoning, so difficulty-adaptive budgets are preferable to static ones. These methods reduce benign reasoning cost and suggest possible controls for the “overthinking” surface exploited by ReasoningBomb, ThinkTrap, OverThink, and ExtendAttack; their robustness against those adaptive attacks remains to be established unless directly evaluated in the cited work. The value of enforceable budgets is that they do not depend on first classifying an input as malicious, which matters because the evaluated detectors exhibit high bypass rates. ReasoningBomb reports bypass rates of 99.8%, 98.7%, and 98.4% for its input-, output-, and joint-detection baselines, respectively [66]. Enforceable budgets therefore provide a complementary guarantee, and this observation supports skepticism toward those specific detectors rather than a universal ranking over all future detection methods.

Among existing defenses, hard runtime enforcement is the most direct way to impose a configured upper bound on admitted computation rather than merely reducing average cost, and we expect it to grow in importance as models expose more input-dependent computation. It is not, however,

a complete or cost-free solution, and presenting it as one would understate its risks. A hard cap can itself induce integrity and fairness failures: detection caps can discard real objects in crowded scenes; token or reasoning caps can truncate legitimate long or hard answers, disproportionately harming difficult-but-valid requests; admission control can deny service to benign heavy users; and a fixed execution budget can open a new timing or policy side channel (the cap boundary is itself observable). Composition is a further weakness: an adaptive attacker can craft inputs that respect every *local* cap while still driving cumulative *end-to-end* overload across stages, so per-stage budgets do not compose into an end-to-end guarantee without a global accounting layer. Budget enforcement is therefore best understood as a risk-controlled trade-off—bounding worst-case resource use at a tunable cost in clean-input utility and fairness—rather than a universal remedy, and its parameters should be set with the attacker-cost and clean-utility terms of Section 2.5 in view.

Table 5. Strength of reasoning-cost controls, from hard runtime enforcement to soft prompt guidance. Only hard runtime enforcement guarantees a configured upper bound; the weaker categories reduce expected cost and require adversarial evaluation. This table refines the “Budget enforcement” rows of Table 6.

Control type	Examples	Guarantee
Hard runtime enforcement	Token cap, termination, admission control [11, 129]	Configured upper bound
Risk-controlled stopping	Conformal Thinking [111]	Statistical guarantee under stated assumptions
Learned length control	CoT-Valve [74]	Empirical length reduction
Prompt guidance	TALE [36], Concise CoT [91]	Soft behavioral instruction

8.2 Robust and adaptive training

Hardware-aware adversarial training [109] builds background attention into the training pipeline using objectness loss as a proxy, restoring real-time throughput from 13 FPS to 43 FPS under attack on a Jetson Orin NX, with a better clean/robust trade-off. For multi-exit models, *calibration-based defenses* [103] (temperature scaling, robust exit calibration) recover roughly 60% of the efficiency lost to slowdown attacks. This subsection is limited to methods that change training, calibration, or model robustness; input-side purification methods appear in Section 8.3.

8.3 Input transformation and purification

For NMS-exploitation attacks, *adaptive input resizing* [93] reduces NMS execution time by 90.18% on an attacked YOLOv5-large model—a defense evaluated directly against a latency objective. A complementary input-transformation lineage operates in the frequency domain. *Feature Distillation* [67] first introduced DNN-oriented defensive quantization in the DCT domain of the JPEG pipeline to rectify adversarial perturbations while preserving benign accuracy; *FenceBox* [85] generalized frequency- and spatial-domain purification into a unified augmentation platform; and *DefQ* [86] specialized this defensive-quantization principle to the *availability* setting, being the first dedicated defense against the early-exit slow-down attack: it statistically characterizes the frequency-domain signature that DeepSloth introduces and applies a DCT-domain quantization table—deployable inside the front-end camera’s existing JPEG pipeline—that strips these perturbations and restores early-exit behavior on multi-exit DNNs (recovering early-exit rates to 85%+ across SDN-ResNet-56, VGG-16, and MobileNet). Together with calibration-based mitigation (Section 8), DCT-domain purification provides a complementary input-side intervention for early-exit slowdown attacks. The

remaining methods in this category were designed and evaluated primarily against *integrity* threats (object-vanishing patches, off-manifold visual perturbations), and we include them as *candidate* latency defenses whose effectiveness against a latency objective is largely *untested* rather than as validated countermeasures. Temporal-consistency and DCT-based methods localize or remove physical patches [76, 90]. Texture-feature denoising has been evaluated through mAP recovery and detection time [60], while latent-diffusion purification projects VLM inputs into a low-dimensional latent space to suppress off-manifold perturbations while preserving semantics [56]. Because a purifier that removes an integrity perturbation *may* also remove a latency-inducing one, these are plausible but unconfirmed for the availability setting: establishing whether denoising a phantom-box or verbose-image perturbation actually restores the benign work level would require evaluation against a latency metric, which these papers do not report.

8.4 Bounded and constant-time execution

Two related defenses should be distinguished. *Constant-time* implementations seek to remove input-dependent timing variation within a specified implementation boundary—removing the targeted operation’s data-dependent timing variation, and with it both the latency-inflation lever and the timing side channel. *Bounded-execution* mechanisms retain some data dependence but cap its maximum extent—for example, by limiting the candidates admitted to NMS or the tokens emitted by a decoder. A cap therefore bounds worst-case work without necessarily producing constant runtime: even under a fixed nominal-operation bound, wall-clock time can still vary through memory access, caching, contention, and hardware effects, and end-to-end runtime can remain data-dependent through preprocessing, memory allocation, batching, or downstream stages. Biton et al. [5] explore constant-time NMS to close the timing-leakage channel they identify. Capping the number of detections admitted to NMS, or the number of tokens a decoder may emit, is instead a bounded-execution instance of the *work budget* abstraction (Section 9): it converts an unbounded worst case into a finite one but does not by itself make execution constant-time. The cost is a potential accuracy ceiling in dense scenes, making the bound a tunable safety–utility parameter.

8.5 Architectural sparsity and efficiency reduction

Shrinking per-step compute reduces an attacker’s headroom. *Model pruning* [41] improves resilience to sponge poisoning (energy overhead $\sim 4\times \rightarrow \sim 1.3\times$) at an accuracy cost. *BlindSight* [97] exploits attention sparsity to cut VLM FLOPs by 32–41% with -2% to $+2\%$ accuracy; it is a general efficiency mechanism, not a demonstrated latency-robustness defense. These reduce absolute latency impact but do not *bound* generation length and have not been evaluated against adaptive latency attacks, so they complement rather than replace budget enforcement.

8.6 Runtime monitoring and adaptive inference

Activation-sparsity monitoring offers a lightweight, architecture-agnostic anomaly signal for sponge inputs [77]. For early-exit systems, online scheduling treats the exit decision as a contextual bandit [49] (improving overall performance by up to 98.1% over the best static benchmark), and quorum-based ensemble stopping (SQUAD) [28] reduces latency by up to 70.6% through quorum-based stopping and may reduce dependence on any single confidence score; its robustness against adaptive slowdown attacks has not been established here.

8.7 System-level and serving control

At the serving layer, request scheduling, admission control, and pool right-sizing [11, 129] can reduce, isolate, or bound the effect of operational bottlenecks becoming attack amplifiers, depending on the admission, scheduling, and resource-accounting guarantees they provide. The following

methods are included to identify possible systems primitives or boundary cases, not as validated latency-attack defenses. *Certifiers*—formal robustness verifiers with abstain/fallback behavior—can themselves create an availability surface, since an adversary can systematically trigger abstention to deny service [68]; a safety fallback can become a target. Beyond admission control, hardware-level system defenses target the serving boundary directly: *Garrison* [113] is a GPU-accelerated inference system for adversarial ensemble defense that uses Multi-Instance GPUs with reinforcement-learning-based scheduling, improving adversarial robustness by up to 24.5% while accelerating ensemble inference by 6.6×; its evaluation targets integrity-oriented ensembles, so accelerating the defense—rather than demonstrated protection against latency amplification—is the relevant contribution here. *PSML* [92] secures model-serving systems against model *extraction* via fingerprinting and noise-based defenses; it is adjacent serving security rather than a latency-attack defense, and we list it only to delineate the serving-security boundary. The *time-traveling* defense [22] is an unusual input-side variant for the perception domain: it queries historical Street View imagery and uses majority-vote inference to resist adversarial manipulation of traffic signs, achieving 100% defense effectiveness in its integrity evaluation; it has not been evaluated against a latency objective, and we include it as evidence that temporal redundancy can substitute for per-frame robustness in geographically static scenes.

To distinguish demonstrated defenses from plausible transfers, Table 6 assigns each method an explicit evidence class. *Direct latency*, *direct energy*, *direct compute/work*, and *direct capacity* indicate evaluation against measured latency or slowdown, energy consumption, computational work, or serving capacity or goodput, respectively. These direct classes identify the availability-related quantity evaluated; they do not imply robustness against adaptive latency attacks. *Budget-control* indicates evaluation of compute-budget control without necessarily demonstrating protection against an adaptive latency attack. *Timing channel* indicates evaluation against timing leakage or timing variation with an availability implication. *Integrity only* identifies candidate latency defenses evaluated only against integrity attacks; *general efficiency* identifies efficiency improvements not evaluated against latency attacks; *conceptual* denotes a proposed control without evaluation against an adaptive latency attack; and *adjacent security* denotes a serving or systems-security mechanism not evaluated against latency amplification.

Section Summary. The reviewed controls span several evidence classes: demonstrated latency-attack defenses, candidate transfers from integrity security, general efficiency mechanisms, and conceptual or adjacent system controls (Table 6). None provides comprehensive protection, and many were never evaluated against an adaptive latency attack. Among the demonstrated defenses, most reduce average computational cost or detect anomalous behavior without explicitly bounding worst-case execution. In contrast, mechanisms that directly constrain computation—such as token budgets, bounded execution, and constant-time implementations—address the underlying cause of latency inflation rather than its symptoms. This observation motivates the broader *work-budget abstraction* developed in the next section.

9 Cross-Domain Synthesis and Evaluation

Viewing the perception and LLM/VLM-serving literatures side by side yields three observations that neither community arrives at independently.

Most instantiated perception defenses do not transfer directly to serving, and vice versa. The non-transfer is at the level of concrete mechanisms, not of the abstract principles (budgets, admission control, bounded execution, monitoring) they instantiate. Hardware-aware adversarial training [109], input purification [56, 90], and the time-traveling majority vote [22] all assume the attack arrives through a *sensor input* as a pixel-level perturbation; they have no lever on prompt-injection DoS

Table 6. Defense families against latency and energy attacks, organized by control mechanism and evidence class. The evidence class identifies the quantity or security property evaluated; it does not imply robustness against adaptive latency attacks.

Family	Representative method	Evidence	Reported result
Budget enforcement	PD3F [129]	Direct capacity	Up to 500% access-capacity improvement under adversarial load
Budget enforcement	Token-budget routing [11]	Budget-control	Avoids 4–8× worst-case over-provisioning (also serving control)
Budget enforcement	Conformal Thinking [111]	Budget-control	Risk-controlled reasoning stop (budget-control evaluation, not adaptive-latency)
Robust / adaptive training	Can’t Slow Me Down [109]	Direct latency	13 → 43 FPS under attack
Robust / adaptive training	Exit calibration [103]	Direct compute/work	~ 60% of lost efficiency recovered
Input transformation	Adaptive resizing [93]	Direct latency	90.18% reduction in NMS time
Input transformation	DefQ [86]	Direct compute/work	Early-exit rate restored to 85%+
Input transformation	Feature Distillation [67]	Integrity only	Candidate; DCT-domain purification
Input transformation	ADAV [76]	Integrity only	Candidate; patch localization
Input transformation	DCT purification [90]	Integrity only	Candidate; frequency-domain
Input transformation	LDP [56]	Integrity only	Candidate; latent purification
Input transformation	Time-traveling [22]	Integrity only	Candidate; temporal redundancy
Bounded/constant-time execution	Constant-time NMS [5]	Timing channel	Removes the targeted input-dependent NMS timing variation within the analyzed implementation boundary
Bounded/constant-time execution	Detection / token caps	Conceptual	Proposed work-budget control
Architectural sparsity	Pruning [41]	Direct energy	Attack-induced energy overhead reduced from ~4× to ~1.3× (accuracy cost)
Architectural sparsity	BlindSight [97]	General efficiency	32–41% FLOP reduction
Runtime monitoring & adaptive control	Sparsity monitor [77]	Direct compute/work	Anomaly signal for sponge inputs
Runtime monitoring & adaptive control	DEE [49]	General efficiency	Online exit scheduling
Runtime monitoring & adaptive control	SQUAD [28]	General efficiency	Up to 70.6% latency reduction
System / serving control	Garrison [113]	Adjacent security	Up to +24.5% robustness; 6.6× inference acceleration
System / serving control	PSML [92]	Adjacent security	Model-extraction defense
System / serving control	Certifier caveat [68]	Conceptual	Abstain/fallback can create an availability surface

Notes. Reported improvements use each source paper’s own baseline and are not directly comparable across rows.

or output-length inflation, where the malicious content is carried in the semantics of a query rather than in pixel noise. Conversely, PD3F [129], token-budget routing [11], and conformal stopping [111] assume a serving stack with request queues and preemptible generation, which has no analog in a camera-to-detector pipeline. This mechanism-level non-transfer is the central practical obstacle to securing a heterogeneous autonomy stack, even where a common design principle applies.

Intermediate-work amplification recurs, and suggests a shared class of remedies. As argued in Section 2.3, most of the attack classes we survey derive their leverage from forcing a downstream stage to receive more intermediate objects than it was provisioned to process. Where that shape holds, a natural cross-domain abstraction is a *work budget*: an explicit limit on the number of intermediate objects admitted to a stage whose cost rises with that count. We are careful not to overclaim its reach: it fits the count-driven attacks well (NMS overload, token/step inflation, expert routing), but it does not by itself address hardware fault injection or timing side channels, which are governed by different mechanisms (Section 7).

We view the work budget as an abstract design principle rather than a concrete algorithm. Existing mechanisms—including constant-time NMS [5], detection caps, token budgets [11], and risk-controlled reasoning stops [111]—can be interpreted through this common perspective. They differ in guarantee strength: hard caps and constant-time implementations directly bound or fix execution within a declared boundary, whereas routing and statistical stopping mechanisms reduce or control expected cost without necessarily establishing an adversarial worst-case bound. Viewed in this way, latency defenses become less about protecting individual models and more about controlling computation throughout the inference pipeline. This is a design stance, not a solved problem: as we detail below, every such cap trades worst-case cost against clean-input utility and fairness, and a determined attacker can stay just under each local cap while still driving cumulative end-to-end overload. Figure 3 illustrates the work-budget abstraction with perception and serving examples.

No evaluated end-to-end defense. In this survey, *end-to-end evaluation* means an adversarial workload propagated through at least two heterogeneous computational stages, with both resource consumption and a terminal system outcome measured (for autonomous systems, e.g., deadline misses, collision rate, control degradation, or mission completion; for serving, e.g., goodput, victim latency, admission failure, or completed useful requests). Within the corpus we reviewed, each documented defense (Table 6) protects one layer and leaves others exposed; we did not identify a defense evaluated end-to-end in this sense across a multi-stage pipeline. A VLM-equipped autonomous vehicle (AV) faces simultaneous latency threats at the camera-to-detector boundary (NMS overload, physical projectors), the tracking layer (SlowTrack-style temporal attacks), the LiDAR and cooperative-fusion stages, and the VLM planner’s decoder (verbose images, reflection backdoors). Composing defenses across these layers is technically possible but, to our knowledge and within our search, has not been evaluated end-to-end—a gap we return to below.

9.1 Evaluation and benchmarking

Progress is hampered by fragmented evaluation. Perception-attack papers use datasets such as MOT17, nuScenes, and KITTI, report wall-clock latency on specific GPUs (frequently an NVIDIA 2080 Ti or a Jetson device), and—in the strongest cases—report system-level outcomes on the Baidu Apollo stack and LGSVL simulator [72]. LLM/VLM-serving evaluations instead use vLLM-style serving benchmarks on models ranging up to Llama-3-70B, and report access capacity, token amplification, or detector-bypass rate [66, 129]. Three deficiencies recur. First, there is *no standardized*

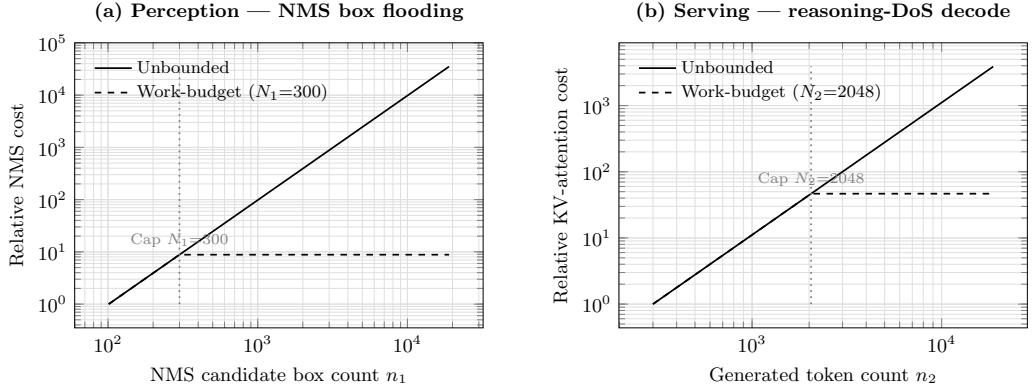


Fig. 3. Conceptual illustration of the work-budget abstraction across perception and serving systems. Limiting the objects admitted to a stage bounds that stage’s count-dependent work, assuming the cap is applied before the expensive operation; it does not by itself bound upstream candidate generation, pre-cap sorting/selection, memory allocation and copying, other uncapped stages, or cumulative work across repeated tool/agent calls. The left panel illustrates a candidate cap applied against a Groundswell-style NMS-overload workload [116] and distinguishes *generated* candidates from candidates *admitted* to NMS; the right panel illustrates a generation budget applied against a ReasoningBomb-style reasoning-inflation workload [66] and distinguishes attack-induced continuation *pressure* from tokens *actually emitted* before termination. These citations identify motivating attacks, not the depicted cap implementations. Both panels use log-log axes; the capped curve (dashed) plateaus at the vertical cap line while the unbounded curve (solid) keeps rising. The figure is conceptual and intended to illustrate the common design principle underlying several existing defenses.

latency threat model analogous to the ℓ_p -ball used for evasion: papers leave attacker access, perturbation/query budget, and the target latency metric unspecified or mutually inconsistent. Second, there is *no agreed success metric*—wall-clock time, FPS, FLOPs, energy, crash rate, and access capacity are used interchangeably, defeating cross-paper comparison. Third, *no benchmark jointly stresses perception and serving* under attack, which is precisely what an end-to-end AD-plus-VLM stack would require. We argue that joint latency/robustness protocols—measuring accuracy and timing *simultaneously* under adversarial load—are a prerequisite for trustworthy comparison, because efficiency and robustness interact non-monotonically (Section 10).

Toward standardization, we suggest that a latency-attack evaluation should report along five complementary dimensions, by analogy to the way evasion benchmarks fixed attacker knowledge, perturbation norm, and success rate. (i) *Threat model*: the attacker’s interface (input, prompt, weights, hardware), knowledge (white/grey/black-box), and query/perturbation budget, stated explicitly rather than left implicit. (ii) *Cost metric*: at least one *implementation-independent* work proxy, where meaningful (FLOPs, decoder steps, admitted candidate boxes, or selected experts), reported alongside any wall-clock or FPS number, since the latter cannot be interpreted or compared reliably without the exact accelerator, batch size, and serving framework; such proxies still require model and configuration context (context length, attention implementation, cache state, batching, architecture) and should not be treated as interchangeable. (iii) *Amplification baseline*: the benign-input cost against which inflation is measured, including whether the baseline is a clean average or a benign worst case, because a 10× figure against an easy baseline may be smaller in absolute terms than a 2× figure against a hard one. (iv) *Downstream impact*: a system-level outcome (deadline-miss rate, collision rate, mission completion, or service access capacity) rather than component

timing alone, since a $5\times$ slowdown that still meets the deadline is operationally harmless while a $1.5\times$ slowdown that misses it is catastrophic. (v) *Defense interaction*: whether the attack was evaluated against any deployed budget, cap, or monitor, since an attack that collapses under a trivial detection cap is qualitatively different from one that survives it. Adopting these reporting dimensions in future evaluations would make results more comparable than the heterogeneous native measurements summarized in Tables 3 and 4; a reporting protocol cannot retroactively make already-published native metrics directly comparable. We stress that no single number captures a latency attack: an attack is characterized by a *cost-impact pair* under a stated threat model, and reporting one without the other is the single most common evaluation gap in the literature we surveyed.

Section Summary. Viewed through this lens, latency attacks are less about individual algorithms than about uncontrolled computational growth. Although different application domains expose different bottlenecks, many attack classes repeatedly exhibit intermediate-work amplification, while hardware faults, timing leakage, and some availability-adjacent embodied attacks require separate mechanisms. The work-budget abstraction provides a unified perspective on existing defenses and suggests that future research should focus on bounding computation across heterogeneous AI systems rather than protecting individual models in isolation.

10 Emerging Surfaces and Open Challenges

10.1 NMS-free detectors and moving bottlenecks

A salient shift affects much of the object-detection attack literature: modern detectors increasingly remove NMS. YOLOv10 [105] and DETR-style models [8] replace NMS with one-to-one label assignment and lightweight decoder heads, removing the data-dependent quadratic post-processing that Phantom Sponges and Overload exploit. It is important to be precise about what this changes. Standard DETR uses Hungarian bipartite matching during *training*, where predictions are matched to ground-truth objects to compute the loss; ordinary inference has no ground-truth set and therefore does not run Hungarian matching as a per-image inference step. Any claim that adversarial inputs inflate Hungarian iterations during ordinary DETR inference is therefore incorrect unless a specific deployed implementation performs assignment at inference time, and the $O(N^3)$ matching cost cannot be presented as a general inference-time attack surface.

Removing NMS does not by itself eliminate availability risk; it relocates and reshapes the cost, and the residual surfaces must be established by architecture-specific profiling rather than inferred from training-time behavior. For fixed-query DETR variants, the number of decoder queries and layers is typically fixed, which limits input-driven algorithmic work amplification in the core inference graph. Residual risk is more likely in *dynamic* variants that prune tokens or queries, use adaptive decoder depth, process variable-resolution inputs, invoke downstream tracking or association stages, or exhibit input-dependent memory and hardware behavior. In assessing such risks it is useful to separate three notions: *work amplification*, where the input changes admitted operation or intermediate counts; *latency amplification*, where the input changes measured elapsed time; and *timing variation*, where elapsed time changes without necessarily changing nominal operation counts (through memory access, cache behavior, kernel selection, dynamic preprocessing shapes, contention, thermal state, or downstream tracking). A fixed-query detector can still exhibit input-correlated *timing variation* even when its computational graph is fixed, so an input-dependent wall-clock reading does not by itself imply an input-dependent operation count. We list several *unvalidated, speculative* hypotheses for such surfaces, which we flag explicitly as research conjectures rather than demonstrated attacks: the multi-scale feature-aggregation *neck* (PANet/BiFPN) might be forced to process high-entropy features through all

pyramid levels; *deformable-attention* sampling might be steered toward non-coalesced memory-access patterns; and dynamic token/query pruning might be prevented from pruning. Each of these is a candidate for the kind of profiling study that would be needed to move it from conjecture to a measured vulnerability. We highlight this as an under-explored direction: whether latency threats persist across an architectural change that was *not* motivated by security remains an open, empirically unresolved question.

10.2 Cross-cutting open challenges

Standardized, system-level benchmarks. Most attacks report per-component timing in incompatible units; few connect to downstream outcomes. SlowTrack’s crash-rate evaluation [72] and CP-FREEZER’s testbed [106] are exceptions. The field needs benchmarks that report *both* timing and safety/serviceability impact on common axes.

Physical realizability. Projector attacks [73, 78] demonstrate multi-second physical delays but require active infrastructure; SlowTrack’s physical transfer remains preliminary [72]. The gap between strong digital results and robust physical deployment is wide.

Inference-time compute as a new surface. Recent reasoning models and MoE routers [24, 59, 66] have rapidly expanded existing availability-attack classes and introduced new mechanism-specific variants. As architectures continue to make compute input-dependent, each such mechanism is a candidate target, and defenses lag behind by construction.

Timing as an oracle. Black-box timing attacks [79] show that even opaque deployments leak optimization signals, complicating any defense that assumes the attacker’s ignorance. This challenge deepens as serving systems adopt data-dependent optimizations: speculative decoding, continuous batching, prefix caching, and adaptive quantization all make wall-clock behavior a function of the input, so the same mechanisms that reduce average latency also broaden the timing side-channel. The speculative-decoding fingerprinting result [99]—which recovers user queries and datastore contents from per-iteration token counts—shows that the availability and confidentiality axes are not independent: an optimization added purely for throughput creates input-dependent timing and a demonstrated side-channel surface. Whether an adversary can reliably force draft-token rejection patterns that produce material slowdown beyond a non-speculative baseline remains a separate empirical question: rejecting all draft tokens removes the speedup but need not yield latency worse than ordinary non-speculative decoding, and the cited leakage result does not establish attacker control over rejection. A general design principle follows: any component whose runtime varies with input content should be treated as both an attack target and an information source, and should be evaluated under a threat model that grants the adversary timing observations by default rather than assuming an opaque black box.

The efficiency–robustness trade-off. Input-adaptive efficiency mechanisms—including early exit, dynamic sparsity, token pruning, routing, and speculative decoding—create a gap between average and worst-case computation that an adversary may attempt to exploit; static pruning and quantization can reduce absolute cost but do not by themselves guarantee robustness against adaptive work inflation, and the interaction is non-monotone [41, 101]. Defenses that bound admitted cost by construction (budget enforcement, runtime budget or deadline guards) are more promising than those that merely reduce average cost; note that such a runtime guard bounds admitted execution but does not by itself establish the system’s WCET, which is a stronger analysis-based claim. The deeper tension is architectural: the field has spent a decade designing systems—early-exit networks, token pruning, MoE routing, speculative decoding, KV-cache reuse—whose entire value proposition is that *average* cost falls below worst-case cost. Each such gap is a *potential* attack

surface—headroom an availability attacker may reclaim—but exploitation requires an attacker-controllable relationship between the input and the control path, since data dependence alone does not establish exploitability. This suggests that availability-hardened deployment may require explicitly co-designing the efficiency mechanism with its worst-case bound—for example, an early-exit network that guarantees a maximum depth per unit time regardless of confidence, or a router that caps the fraction of tokens sent to any single expert. Treating the worst-case bound as a first-class design objective, rather than a property to be measured after the fact, is a promising but largely unexplored direction.

Embodied and agentic amplification. The migration of latency attacks from perception front-ends to Vision-Language-Action policies (Section 4.5) and to tool-using LLM agents (Section 6) changes both the consequence and the amplification factor. In an embodied setting, computation-induced deadline misses can propagate directly into physical control consequences—a collision, a dropped payload, a frozen manipulator—but this relationship must be demonstrated under a specified closed-loop stack, deadline, and fallback policy rather than assumed. In an agentic setting, a single injected message can trigger further tool invocation across many nodes, so work can compound across invocations—additively in sequential chains and potentially multiplicatively under recursive or fan-out execution; reported per-message amplifications already reach two orders of magnitude [61, 132], illustrating large system-level amplification without alone proving multiplicative scaling. Both settings break the implicit assumption, common to most existing defenses, that the unit of protection is a single forward pass; defenses will instead have to reason about compute budgets spanning multiple models, multiple invocations, and shared infrastructure.

Detection fragility. High detection-bypass rates [66] argue that anomaly detection alone is insufficient; enforceable budgets and scheduling must complement it. The structural problem is that several attacks preserve the source paper’s reported task metric—BLEU, mAP, or perplexity—so monitors based only on that metric may fail to detect them, leaving the resource footprint as the main observable; note, however, that extremely repetitive or truncated outputs can visibly degrade practical utility even when the benchmark metric is stable. But the resource footprint of a benign hard input (a crowded scene, a genuinely long reasoning problem) can be indistinguishable from that of an adversarial one, forcing any detector into a false-positive/false-negative trade-off that the attacker can tune against. This is why we repeatedly argue for enforcement over detection: a work budget does not need to decide whether an input is malicious, only to cap the cost it is permitted to incur, side-stepping the detection problem entirely at the price of a tunable utility ceiling in genuinely hard cases.

End-to-end stack evaluation. A modern autonomy stack with a VLM planner is simultaneously vulnerable at multiple stages (Section 9), yet within our corpus we did not find work that evaluates all of these jointly. Building a simulation environment that injects attack stimuli at each layer and measures *system-level* outcomes—collision rate, mission completion, service availability—is, in our view, a central open engineering problem in this space. A related open question follows from the defense non-transfer documented in Section 9: whether a single unifying primitive—work-budget enforcement at every stage boundary—can subsume today’s narrowly scoped defenses without unacceptable accuracy cost.

Collectively, these challenges indicate that latency attacks are transitioning from isolated algorithmic curiosities to a fundamental systems-security problem. Addressing them will likely require tighter integration between machine learning, systems, hardware, and real-time computing than current research communities typically consider.

10.3 Toward whole-system availability

The attacks surveyed above were, until recently, comfortably decomposable: an availability attack targeted one model—a detector, a translator, an LLM—and the analysis stopped at that model’s output. Two developments dissolve this boundary. First, *embodied* foundation models (Section 4.5) place a Vision-Language-Action policy in a physical control loop, so the terminal effect of inflated computation is no longer a late tensor but a late *action*—a manipulator that freezes, a vehicle that brakes a beat too slowly. Second, *agentic* systems (Section 6) compose many model invocations, tools, and guardrails into a graph, so a single adversarial input propagates and amplifies across components rather than terminating at one decoder. In both cases the meaningful unit of availability is no longer a forward pass but the *whole system*: a pipeline of heterogeneous models with a shared, finite, real-time compute budget.

This reframing has three consequences for how the field should proceed. *The consequence model changes*. A service-level objective (tokens-per-second, tail latency) is the right currency for a cloud LLM, but embodied systems may impose hard or firm real-time constraints, depending on whether a late result is unsafe, useless, or merely degraded. In safety-critical embodied systems a deadline miss may become a safety-relevant event depending on the control period, slack, fallback policy, and closed-loop dynamics, rather than being a safety event automatically. Latency attacks on embodied systems therefore belong as much to the safety and real-time-systems literature as to adversarial machine learning, and evaluating them demands the closed-loop, outcome-level metrics (collision rate, task completion) that Section 9.1 argues are still missing. An observed adversarial maximum is not a worst-case execution time (WCET) bound: establishing WCET requires a sound analysis over the declared input, software, scheduling, and hardware state spaces, which is especially delicate on GPUs where contention, caches, dynamic frequency scaling, and asynchronous kernels complicate timing guarantees. *The amplification model changes*. In a single model, amplification is bounded by one architectural mechanism—box count, token count, expert load. In an agent graph, amplification is governed by the executed graph’s depth, fan-out, recursion, retry policy, and termination limits. For an invocation graph $G = (V, E)$ with executed node set V_{exec} , the total induced work is additive over executed invocations,

$$W_{\text{graph}} = \sum_{v \in V_{\text{exec}}} W_v, \quad (3)$$

so what may grow is the *number* of executed invocations: with branching factor greater than one and without deduplication or execution caps, $|V_{\text{exec}}|$ may grow exponentially with effective depth. It is therefore useful to distinguish four forms of amplification:

- *Sequential amplification*—the executed graph depth grows;
- *Fan-out amplification*—the number of executed nodes grows by branching;
- *Recursive amplification*—repeated cycles or retries expand the execution trace;
- *Resource amplification*— W_{graph} relative to matched benign graph work.

Under this accounting the same intermediate-work amplification principle can compound across invocations—additively in per-node work but with the node count able to grow under recursion or fan-out; the two-order-of-magnitude per-message figures reported for agentic denial-of-service [61, 132] illustrate large system-level amplification, and guardrails themselves become amplifiers rather than mitigations when they can be trapped in extended reasoning [133]. The compounding can also be *contagious*: CORBA [134] shows that a single benign-looking, recursively self-propagating instruction can spread across a multi-agent system, driving cycles of meaningless message passing that exhaust compute cluster-wide—an availability failure whose blast radius is the agent topology itself rather than any one model. Emerging security taxonomies for autonomous agents make the

same point structurally: the Hierarchical Autonomy Evolution framework [126] places resource-monopolization and computational denial-of-service at its *collective-autonomy* tier, where a single node’s worst-case computation propagates through the multi-agent topology as a cascade failure—precisely the systemic, cross-component amplification that a per-model budget cannot see. *The defense model changes.* A per-model cap—bounded NMS, a token budget, a reasoning limit—is necessary but no longer sufficient, because an adversary can respect every local budget while still exhausting a shared global one through breadth. The natural generalization of the work-budget abstraction (Section 9) is therefore a *hierarchical* budget: local caps at each stage boundary, composed under a global admission-control layer that accounts for the total intermediate work a single request may induce across the entire graph, and that degrades gracefully—shedding or deferring low-priority work—rather than failing when the budget is approached.

We therefore argue that whole-system availability is emerging as the organizing problem for the next phase of this research. It subsumes the perception/serving non-transfer (Section 9), the missing end-to-end benchmark (Section 9.1), and the embodied/agentive amplification challenge (Section 10) under a single question: *Can a heterogeneous, multi-model, real-time AI system guarantee a bounded worst-case response under adversarial load?* We identified no work in the reviewed corpus that provides such an end-to-end adversarial worst-case guarantee, and we believe progress will come less from hardening individual models than from treating the compute budget of the entire pipeline as the asset to be defended.

11 Conclusion

This survey advances a single organizing idea: many latency attacks, across object detection, adaptive networks, autoregressive generation, reasoning models, and agentive systems, are instances of *intermediate-work amplification*, and the natural cross-domain defense is to bound that work through *work-budget enforcement*. This attack-to-defense arc gives a unified view of a literature that has until now been fragmented, and it is the lens we recommend for organizing future work on availability.

Latency attacks extend the security of deep learning beyond correctness to timeliness and resource availability. What began as NMS overload on a single detector has matured into a crosscutting threat spanning physical autonomous-driving perception, sponge attacks on accelerators, slowdown attacks on every form of conditional computation, and a rapidly expanding frontier of output-length, verbose-image, and reasoning denial-of-service attacks on LLMs and VLMs. A recurring lesson is that any mechanism which makes inference cost input-dependent—post-processing, sparsity, early exit, routing, autoregressive generation, inference-time reasoning—can create an attack surface once efficiency is advertised. Data dependence alone does not establish exploitability. Even so, many prominent exploited surfaces arise from mechanisms originally introduced for efficiency, including sparsity, adaptive depth, pruning, routing, caching, batching, and speculative decoding. Hardware faults, training-data and corpus poisoning, physical projector delivery, guardrail abuse, and self-propagating agent instructions are exceptions that do not fit this pattern. Defenses are advancing across robust training, input transformation, runtime monitoring, and serving control. Among these, budget enforcement and bounded execution most directly constrain high-cost computation; evidence for many purification and general-efficiency methods against adaptive latency attacks remains limited (Section 8). They also remain fragmented and bound by a persistent efficiency–robustness trade-off. Progress will depend on standardized benchmarks that report timing alongside downstream impact, on threat models that take physical and serving realism seriously, and on defenses that control high-cost computational paths rather than merely lowering the average. As deep learning continues to trade fixed cost for input-adaptive efficiency, the availability axis will only grow in importance. We expect availability-oriented attacks to become a permanent component of

adversarial machine learning research rather than a niche topic. As future architectures increasingly expose dynamic computation, understanding worst-case inference behavior will likely become as important as improving average-case efficiency. Ultimately, availability is evolving from a property of individual models into a property of the entire AI system, and ensuring bounded computation may become as fundamental to trustworthy AI as ensuring predictive accuracy.

References

- [1] Francisco Aguilera-Martínez and Fernando Berzal. 2025. LLM Security: Vulnerabilities, Attacks, Defenses, and Countermeasures. arXiv preprint arXiv:2505.01177. doi:10.48550/arXiv.2505.01177
- [2] Muhammad Shakeel Akram, Vincent Meyers, Mehdi B. Tahoori, Bogaraju Sharatchandra Varma, and Dewar Finlay. 2025. EvoWeight: Sponge Poisoning of FPGA-Based DNN Accelerators in Differentially Private Secure Federated Learning. In *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, Piscataway, NJ, USA, 1–11. doi:10.1109/HOST64725.2025.11050058
- [3] Mohammed Ayyat and Tamer Nadeem. 2025. Tricking Dynamic Neural Networks on the Edge: Adversarial Attacks Targeting System Resources. SSRN Electronic Journal, preprint 6829890. doi:10.2139/ssrn.6829890
- [4] Amit Baras, Alon Zolfi, Yuval Elovici, and Asaf Shabtai. 2025. QuantAttack: Exploiting Quantization Techniques to Attack Vision Transformers. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Tucson, AZ, USA, 6730–6740. arXiv:2312.02220 doi:10.48550/arXiv.2312.02220
- [5] Dudi Biton, Aditi Misra, Efrat Levy, Jaidip Kotak, Ron Bitton, Roei Schuster, Nicolas Papernot, Yuval Elovici, and Ben Nassi. 2023. The Adversarial Implications of Variable-Time Inference. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISeC@CCS)*. ACM, New York, NY, USA, 103–114. doi:10.1145/3605764.3623912
- [6] Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. Bad Characters: Imperceptible NLP Attacks. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*. IEEE, San Francisco, CA, USA, 1987–2004. doi:10.1109/SP46214.2022.9833641
- [7] Hanane F. Z. Brachemi Meftah, Wassim Hamidouche, Sid Ahmed Fezza, and Olivier Deforges. 2026. Energy-Latency Attacks: A New Adversarial Threat to Deep Learning. *Comput. Surveys* 58, 8, Article 203 (Feb. 2026), 34 pages. doi:10.1145/3785666
- [8] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-End Object Detection with Transformers. In *European Conference on Computer Vision (ECCV)*. Springer, Cham, Switzerland, 213–229. doi:10.1007/978-3-030-58452-8_13
- [9] Erh-Chung Chen, Pin-Yu Chen, I-Hsin Chung, and Che-Rung Lee. 2024. Overload: Latency Attacks on Object Detection for Edge Devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Piscataway, NJ, USA, 24716–24725. doi:10.1109/CVPR52733.2024.02334
- [10] Hanlin Chen, Simin Chen, Wenyu Li, Wei Yang, and Yiheng Feng. 2025. Impact Analysis of Inference Time Attack of Perception Sensors on Autonomous Vehicles. arXiv preprint arXiv:2505.03850. doi:10.48550/arXiv.2505.03850
- [11] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. 2026. Token-Budget-Aware Pool Routing for Cost-Efficient LLM Inference. arXiv:2604.09613 doi:10.48550/arXiv.2604.09613
- [12] Simin Chen, Mirazul Haque, Cong Liu, and Wei Yang. 2022. DeepPerform: An Efficient Approach for Performance Testing of Resource-Constrained Neural Networks. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, Rochester, MI, USA, 1–13. doi:10.1145/3551349.3561158
- [13] Simin Chen, Cong Liu, Mirazul Haque, Zihe Song, and Wei Yang. 2022. NMTSlloth: Understanding and Testing Efficiency Degradation of Neural Machine Translation Systems. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, USA, 1148–1160. doi:10.1145/3540250.3549102
- [14] Simin Chen, Zihe Song, Mirazul Haque, Cong Liu, and Wei Yang. 2022. NIGCSlowDown: Evaluating the Efficiency Robustness of Neural Image Caption Generation Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Piscataway, NJ, USA, 15365–15374. doi:10.1109/CVPR52688.2022.01493
- [15] Yiming Chen, Simin Chen, Zexin Li, Wei Yang, Cong Liu, Robby T. Tan, and Haizhou Li. 2023. Dynamic Transformers Provide a False Sense of Efficiency. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 7180–7193. doi:10.48550/arXiv.2305.12228
- [16] Antonio Emanuele Cinà, Ambra Demontis, Battista Biggio, Fabio Roli, and Marcello Pelillo. 2022. Energy-Latency Attacks via Sponge Poisoning. arXiv preprint arXiv:2203.08147. doi:10.48550/arXiv.2203.08147
- [17] Zachary Coalson, Gabriel Ritter, Rakesh Bobba, and Sanghyun Hong. 2023. BERT Lost Patience Won't Be Robust to Adversarial Slowdown. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, New Orleans, LA, USA, 1–20. doi:10.48550/arXiv.2310.19152

- [18] Aobotao Dai, Xinyu Ma, Lei Chen, Songze Li, and Lin Wang. 2025. When Data Manipulation Meets Attack Goals: An In-depth Survey of Attacks for VLMs. arXiv preprint arXiv:2502.06390. doi:10.48550/arXiv.2502.06390
- [19] Ali Dehghantanha and Sajad Homayoun. 2026. SoK: The Attack Surface of Agentic AI – Tools and Autonomy. arXiv preprint arXiv:2603.22928. doi:10.48550/arXiv.2603.22928
- [20] Jianshuo Dong, Ziyuan Zhang, Qingjie Zhang, Tianwei Zhang, Hao Wang, Hewu Li, Qi Li, Chao Zhang, Ke Xu, and Han Qiu. 2024. An Engorgio Prompt Makes Large Language Model Babble on. arXiv preprint arXiv:2412.19394. doi:10.48550/arXiv.2412.19394
- [21] Dongfang Du, Chaofeng Sha, and Xin Peng. 2025. Energy Attack Method for Adaptive Multi-Exit Neural Networks. *Information and Software Technology* 179 (2025), 107653. doi:10.1016/j.infsof.2024.107653
- [22] Anthony Etim and Jakub Szefer. 2024. Time Traveling to Defend Against Adversarial Example Attacks in Image Classification. arXiv preprint arXiv:2410.08338. doi:10.48550/arXiv.2410.08338
- [23] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961 doi:10.48550/arXiv.2101.03961
- [24] Zekun Fei, Zihao Wang, Weijie Liu, Ruiqi He, Jianing Geng, Zheli Liu, and XiaoFeng Wang. 2026. Misrouter: Exploiting Routing Mechanisms for Input-Only Attacks on Mixture-of-Experts LLMs. arXiv preprint arXiv:2605.04446. doi:10.48550/arXiv.2605.04446
- [25] Xiaoning Feng, Xiaohong Han, Simin Chen, and Wei Yang. 2022. LLMeffiChecker: Understanding and Testing Efficiency Degradation of Large Language Models. arXiv preprint arXiv:2210.03696. doi:10.48550/arXiv.2210.03696
- [26] Jiyuan Fu, Kaixun Jiang, Lingyi Hong, Jinglun Li, Haijing Guo, Dingkan Yang, Zhaoyu Chen, and Wenqiang Zhang. 2025. LingoLoop Attack: Trapping MLLMs via Linguistic Context and State Entrapment into Endless Loops. arXiv preprint arXiv:2506.14493. doi:10.48550/arXiv.2506.14493
- [27] Xiaowei Fu and Lei Zhang. 2025. Adversarial Defense in Vision-Language Models: An Overview. In *International Conference on Image Processing, Computer Vision and Machine Learning (ICICML)*. IEEE, Piscataway, NJ, USA, 1–6. doi:10.1109/ICICML67980.2025.11333826
- [28] Matteo Gambella, Fabrizio Pittorino, Giuliano Casale, and Manuel Roveri. 2026. SQUAD: Scalable Quorum Adaptive Decisions via Ensemble of Early Exit Neural Networks. arXiv preprint arXiv:2601.22711. doi:10.48550/arXiv.2601.22711
- [29] Kuofeng Gao, Yang Bai, Jindong Gu, Shu-Tao Xia, Philip Torr, Zhifeng Li, and Wei Liu. 2024. Inducing High Energy-Latency of Large Vision-Language Models with Verbose Images. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Online, 1–26. doi:10.48550/arXiv.2401.11170
- [30] Kuofeng Gao, Tianyu Pang, Chao Du, Yong Yang, Shu-Tao Xia, and Min Lin. 2024. Denial-of-Service Poisoning Attacks against Large Language Models. arXiv preprint arXiv:2410.10760. doi:10.48550/arXiv.2410.10760
- [31] Xiaoxue Gao, Yiming Chen, Xianghu Yue, Yu Tsao, and Nancy F. Chen. 2025. TTSlow: Slow Down Text-to-Speech with Efficiency Robustness Evaluations. arXiv preprint arXiv:2407.01927. doi:10.48550/arXiv.2407.01927
- [32] Zeyu Gao, Amin Saremi, and Zonghua Gu. 2026. Performance Measurement and Analysis of Certifiable Defenses against Adversarial Patch Attacks. *Journal of Signal Processing Systems* 98, 1 (2026), 22.
- [33] Soumendu Kumar Ghosh, Arnab Raha, and Vijay Raghunathan. 2023. Energy-Efficient Approximate Edge Inference Systems. *ACM Transactions on Embedded Computing Systems* 22, 4 (2023), 77:1–77:50. doi:10.1145/3589766
- [34] Arijit Ghosh Chowdhury, Md. Mofijul Islam, Vaibhav Kumar, F. H. Shezan, Vinija Jain, and Aman Chadha. 2024. Breaking Down the Defenses: A Comparative Survey of Attacks on Large Language Models. arXiv preprint arXiv:2403.04786. doi:10.48550/arXiv.2403.04786
- [35] Shikha Goel, Rajesh Kedia, Rijurekha Sen, and M. Balakrishnan. 2025. EXPRESS: A Framework for Execution Time Prediction of Concurrent CNNs on Xilinx DPU Accelerator. *ACM Transactions on Embedded Computing Systems* 24, 1 (2025), 11:1–11:31. doi:10.1145/3697835
- [36] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2024. Token-Budget-Aware LLM Reasoning. arXiv preprint arXiv:2412.18547. doi:10.48550/arXiv.2412.18547
- [37] Mirazul Haque, Anki Chauhan, Cong Liu, and Wei Yang. 2020. ILFO: Adversarial Attack on Adaptive Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Seattle, WA, USA, 14264–14273. doi:10.1109/CVPR42600.2020.01428
- [38] Mirazul Haque, Simin Chen, Wasif Haque, Cong Liu, and Wei Yang. 2023. AntiNODE: Evaluating Efficiency Robustness of Neural ODEs. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*. IEEE, Paris, France, 1507–1517. doi:10.1109/ICCVW60793.2023.00163
- [39] Mirazul Haque, Rutvij Shah, Simin Chen, Berrak Sisman, Cong Liu, and Wei Yang. 2023. SlothSpeech: Denial-of-Service Attack Against Speech Recognition Models. In *Proceedings of INTERSPEECH*. ISCA, Dublin, Ireland, 1274–1278. doi:10.21437/Interspeech.2023-2124
- [40] Mirazul Haque, Yaswanth Yadlapalli, Wei Yang, and Cong Liu. 2022. EREBA: Black-box Energy Testing of Adaptive Neural Networks. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, Pittsburgh, PA, USA, 835–846. doi:10.1145/3510003.3510088

- [41] Syed Mhamudul Hasan, Hussein Zangoti, Iraklis Anagnostopoulos, and Abdur R. Shahid. 2025. Sponge Attacks on Sensing AI: Energy-Latency Vulnerabilities and Defense via Model Pruning. [arXiv:2505.06454](#) [doi:10.48550/arXiv.2505.06454](#)
- [42] Sanghyun Hong, Yiğitcan Kaya, Ionuț-Vlad Modoranu, and Tudor Dumitras. 2021. A Panda? No, It's a Sloth: Slowdown Attacks on Adaptive Multi-Exit Neural Network Inference. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Online, 1–23. [arXiv:2010.02432](#) [doi:10.48550/arXiv.2010.02432](#)
- [43] Md. Iqbal Hossain, Neeresh Kumar Perla, Afia Sajeeda, Siyu Xia, and Ming Shao. 2026. A Survey of Recent Advances in Adversarial Attack and Defense on Vision-Language Models. *Neural Networks* 200 (2026), 108764. [doi:10.1016/j.neunet.2026.108764](#)
- [44] Mengjie Hu, Zirui Li, Haotian Wang, Yu Wang, Yufei Ding, Chun Liu, and Qing Song. 2026. Trajectory-Aware Attack: Explainable Adversarial Attack Against Multiple Object Trackers. *IEEE Transactions on Multimedia* 28 (2026), 3284–3299. [doi:10.1109/TMM.2026.3651102](#)
- [45] Benxuan Huang, Lihui Pang, and Anmin Fu. 2024. Sponge Attack against Multi-Exit Networks with Data Poisoning. *IEEE Access* 12 (2024), 33843–33851. [doi:10.1109/ACCESS.2024.3370849](#)
- [46] Ruixuan Huang, Qingyue Wang, Hantao Huang, Yudong Gao, Dong Chen, Shuai Wang, and Wei Wang. 2025. RepetitionCurse: Measuring and Understanding Router Imbalance in Mixture-of-Experts LLMs under DoS Stress. [arXiv preprint arXiv:2512.23995](#). [doi:10.48550/arXiv.2512.23995](#)
- [47] Yiyang Huang, Zixuan Wang, Zishen Wan, Yapeng Tian, Haobo Xu, Yinhe Han, and Yiming Gan. 2025. ANNIE: Be Careful of Your Robots. [arXiv preprint arXiv:2509.03383](#). [doi:10.48550/arXiv.2509.03383](#)
- [48] Yunhan Jia, Yantao Lu, Junjie Shen, Qi Alfred Chen, Hao Chen, Zhenyu Zhong, and Tao Wei. 2020. Fooling Detection Alone is Not Enough: Adversarial Attack against Multiple Object Tracking. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, Online, 1–14. [arXiv:1905.11026](#) [doi:10.48550/arXiv.1905.11026](#)
- [49] Weiwei Ju, Wei Bao, Liming Ge, and Dong Yuan. 2021. Dynamic Early Exit Scheduling for Deep Neural Network Inference through Contextual Bandits. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, New York, NY, USA, 823–832. [doi:10.1145/3459637.3482335](#)
- [50] Donghwa Kang, Doohyun Kim, Sang-Ki Ko, Jinkyu Lee, Hyeongbo Baek, and Brent ByungHoon Kang. 2026. Timestep-Compressed Attack on Spiking Neural Networks through Timestep-Level Backpropagation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 9 pages. [doi:10.48550/arXiv.2508.13812](#)
- [51] Mazaher Kianpour and Shao-Fang Wen. 2019. Timing Attacks on Machine Learning: State of the Art. In *Proceedings of the SAI Intelligent Systems Conference (IntelliSys) (Advances in Intelligent Systems and Computing, Vol. 1037)*. Springer, Cham, Switzerland, 111–125. [doi:10.1007/978-3-030-29516-5_10](#)
- [52] Jiyong Kim, Jaeho Lee, Jiahao Lin, Alish Kanani, Miao Sun, Ümit Y. Ogras, and Jaehyun Park. 2025. eMamba: Efficient Acceleration Framework for Mamba Models in Edge Computing. *ACM Transactions on Embedded Computing Systems* 24, 5s (2025), 110:1–110:22. [doi:10.1145/3762190](#)
- [53] Sarada Krithivasan, Sanchari Sen, and Anand Raghunathan. 2020. Sparsity Turns Adversarial: Energy and Latency Attacks on Deep Neural Networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 11 (2020), 4129–4141. [doi:10.1109/TCAD.2020.3013077](#)
- [54] Sarada Krithivasan, Sanchari Sen, Nitin Rathi, Kaushik Roy, and Anand Raghunathan. 2022. Efficiency Attacks on Spiking Neural Networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*. ACM, New York, NY, USA, 373–378. [doi:10.1145/3489517.3530443](#)
- [55] Abhinav Kumar, Jaechul Roh, Ali Naseh, Marzena Karpinska, Mohit Iyyer, Amir Houmansadr, and Eugene Bagdasarian. 2025. OverThink: Slowdown Attacks on Reasoning LLMs. [arXiv:2502.02542](#) [doi:10.48550/arXiv.2502.02542](#)
- [56] Mengfan Li, Xinxin Fan, Quanliang Jing, Shaoye Luo, Yunfeng Lu, and Jingping Bi. 2025. LDP: Latent Diffusion-based Adversarial Purification towards Transformer-based Visual Encoders. In *Proceedings of the IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, Piscataway, NJ, USA, 532–539. [doi:10.1109/Trustcom66490.2025.00065](#)
- [57] Xingyu Li, Xiaolei Liu, Cheng Liu, Yixiao Xu, Kangyi Ding, Bangzhou Xin, and Jia-Li Yin. 2026. LoopLLM: Transferable Energy-Latency Attacks in LLMs via Repetitive Generation. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence (AAAI)*. AAAI Press, Singapore, 31770–31777. [doi:10.48550/arXiv.2511.07876](#)
- [58] Yu Li, Biao Huang, Jinyin Hu, and Cheng Zhuo. 2026. SPLAT: Revisiting Latency Attack on Dynamic Neural Networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 45, 1 (Jan. 2026), 506–518. [doi:10.1109/TCAD.2025.3576320](#)
- [59] Yunzhe Li, Jianan Wang, Hongzi Zhu, James Lin, Shan Chang, and Minyi Guo. 2026. ThinkTrap: Denial-of-Service Attacks against Black-box LLM Services via Infinite Thinking. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 1–16. [arXiv:2512.07086](#) [doi:10.48550/arXiv.2512.07086](#)
- [60] Jin Liang, Ran Yi, Jie Chen, et al. 2024. Securing Autonomous Vehicles Visual Perception: Adversarial Patch Attack and Defense Schemes with Experimental Validations. *IEEE Transactions on Intelligent Vehicles* 9, 7 (2024), 5234–5247.

- doi:10.1109/TIV.2024.3403667
- [61] Zi Liang, Ronghua Li, Yanyun Wang, Qingqing Ye, and Haibo Hu. 2026. Can a Single Message Paralyze the AI Infrastructure? The Rise of AbO-DDoS Attacks through Targeted Mobius Injection. arXiv preprint arXiv:2605.11442. doi:10.48550/arXiv.2605.11442
- [62] Zhiyu Liao, Kang Chen, Yuanguo Lin, Kangkang Li, Yunxuan Liu, Hefeng Chen, Xingwang Huang, and Yuanhui Yu. 2025. Attack and Defense Techniques in Large Language Models: A Survey and New Perspectives. arXiv preprint arXiv:2505.00976. doi:10.48550/arXiv.2505.00976
- [63] Chengliang Liu, Liangbo Ning, Yujuan Ding, and Wenqi Fan. 2026. Inference Cost Attacks for Retrieval-Augmented Large Language Models. In *Proceedings of the ACM Web Conference 2026 (WWW)*. ACM, Dubai, United Arab Emirates, 7564–7575. doi:10.1145/3774904.3792683
- [64] Han Liu, Yuhao Wu, Zhiyuan Yu, Yevgeniy Vorobeychik, and Ning Zhang. 2023. SlowLiDAR: Increasing the Latency of LiDAR-Based Detection Using Adversarial Examples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Piscataway, NJ, USA, 5146–5155. doi:10.1109/CVPR52729.2023.00498
- [65] Ming Liu, Siyuan Liang, Koushik Howlader, Liwen Wang, Dacheng Tao, and Wensheng Zhang. 2025. Natural Reflection Backdoor Attack on Vision Language Model for Autonomous Driving. arXiv preprint arXiv:2505.06413. doi:10.48550/arXiv.2505.06413
- [66] Xiaogeng Liu, Xinyan Wang, Yechao Zhang, Sanjay Kariyappa, Chong Xiang, Muhao Chen, G. Edward Suh, and Chaowei Xiao. 2026. ReasoningBomb: A Stealthy Denial-of-Service Attack by Inducing Pathologically Long Reasoning in Large Reasoning Models. arXiv:2602.00154 doi:10.48550/arXiv.2602.00154
- [67] Zihao Liu, Qi Liu, Tao Liu, Nuo Xu, Xue Lin, Yanzhi Wang, and Wujie Wen. 2019. Feature Distillation: DNN-Oriented JPEG Compression Against Adversarial Examples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Long Beach, CA, USA, 860–868. doi:10.1109/CVPR.2019.00095
- [68] Tobias Lorenz, Marta Kwiatkowska, and Mario Fritz. 2021. Certifiers Make Neural Networks Vulnerable to Availability Attacks. arXiv preprint arXiv:2108.11299. doi:10.48550/arXiv.2108.11299
- [69] Xiangzhong Luo, Di Liu, Hao Kong, Shuo Huai, Hui Chen, Guochu Xiong, and Weichen Liu. 2024. Efficient Deep Learning Infrastructures for Embedded Computing Systems: A Comprehensive Survey and Future Envision. *ACM Transactions on Embedded Computing Systems* 24, 1 (2024), 1–100. doi:10.1145/3701728
- [70] Zhi Luo, Zenghui Yuan, Wenqi Wei, Daizong Liu, and Pan Zhou. 2025. An Image Is Worth Ten Thousand Words: Verbose-Text Induction Attacks on VLMs. arXiv preprint arXiv:2511.16163. doi:10.48550/arXiv.2511.16163
- [71] Chenxiang Ma, Ningfei Wang, Qi Alfred Chen, and Chao Shen. 2023. WIP: Towards the Practicality of the Adversarial Attack on Object Tracking in Autonomous Driving. In *Symposium on Vehicle Security and Privacy (VehicleSec)*. Internet Society, Reston, VA, USA, 1–8. doi:10.14722/vehiclesec.2023.23063
- [72] Chen Ma, Ningfei Wang, Qi Alfred Chen, and Chao Shen. 2024. SlowTrack: Increasing the Latency of Camera-Based Perception in Autonomous Driving Using Adversarial Examples. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 4062–4070. doi:10.1609/aaai.v38i5.28200
- [73] Chen Ma, Ningfei Wang, Zhengyu Zhao, Qi Alfred Chen, and Chao Shen. 2024. SlowPerception: Physical-World Latency Attack against Visual Perception in Autonomous Driving. arXiv preprint arXiv:2406.05800. doi:10.48550/arXiv.2406.05800
- [74] Xinyin Ma, Guangnian Wan, Runpeng Yu, Gongfan Fang, and Xinchao Wang. 2025. CoT-Valve: Length-Compressible Chain-of-Thought Tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. ACL, Vienna, Austria, 6025–6035. doi:10.18653/v1/2025.acl-long.300
- [75] Vicent Sanz Marco, Ben Taylor, Zheng Wang, and Yehia Elkhatib. 2020. Optimizing Deep Learning Inference on Embedded Systems Through Adaptive Model Selection. *ACM Transactions on Embedded Computing Systems* 19, 1 (2020), 2:1–2:28. doi:10.1145/3371154
- [76] Jiyang Mu. 2024. A Real-Time Defense against Object Vanishing Adversarial Patch Attacks for Object Detection in Autonomous Vehicles. arXiv preprint arXiv:2412.06215. doi:10.48550/arXiv.2412.06215
- [77] Andreas Müller and Erwin Quiring. 2024. The Impact of Uniform Inputs on Activation Sparsity and Energy-Latency Attacks in Computer Vision. In *Proceedings of the IEEE Security and Privacy Workshops (SPW)*. IEEE, Piscataway, NJ, USA, 104–111. doi:10.1109/SPW63631.2024.00016
- [78] Raymond Muller, Ruoyu Song, Chenyi Wang, Yuxia Zhan, Jean-Philippe Monteui, Yanmao Man, Ming Li, Ryan Gerdes, Jonathan Petit, and Z. Berkay Celik. 2025. Investigating physical latency attacks against camera-based perception. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. IEEE, Piscataway, NJ, USA, 18 pages. doi:10.1109/SP61157.2025.00236
- [79] Tsunato Nakai, Daisuke Suzuki, Fumio Omatsu, and Takeshi Fujino. 2021. Adversarial Black-Box Attacks with Timing Side-Channel Leakage. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* 104, 1 (2021), 143–151. doi:10.1587/transfun.2020CIP0022

- [80] K. L. Navaneet, Soroush Abbasi Koohpayegani, Essam Sleiman, and Hamed Pirsiavash. 2024. SlowFormer: Adversarial Attack on Compute and Energy Consumption of Efficient Vision Transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Seattle, WA, USA, 24786–24797. arXiv:2310.02544 doi:10.48550/arXiv.2310.02544
- [81] OWASP Foundation. 2025. *LLM10:2025 Unbounded Consumption*. Technical Report. OWASP Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/> Version 2025.
- [82] Jianhong Pan, Lin Geng Foo, Qichen Zheng, Zhipeng Fan, Hossein Rahmani, Qihong Ke, and Jun Liu. 2023. GradMDM: Adversarial Attack on Dynamic Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 9 (2023), 11374–11381. doi:10.1109/TPAMI.2023.3263183
- [83] Jianhong Pan, Qichen Zheng, Zhipeng Fan, Hossein Rahmani, Qihong Ke, and Jun Liu. 2022. GradAuto: Energy-Oriented Attack on Dynamic Neural Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, Tel Aviv, Israel, 637–653. doi:10.1007/978-3-031-19772-7_37
- [84] Souvik Paul and Nicolas Kourtellis. 2023. Poster: Sponge ML Model Attacks of Mobile Apps. In *Proceedings of the 24th International Workshop on Mobile Computing Systems and Applications (HotMobile)*. ACM, New York, NY, USA, 139–139. doi:10.1145/3572864.3581586
- [85] Han Qiu, Yi Zeng, Tianwei Zhang, Yong Jiang, and Meikang Qiu. 2020. FenceBox: A Platform for Defeating Adversarial Examples with Data Augmentation Techniques. arXiv preprint arXiv:2012.01701. doi:10.48550/arXiv.2012.01701
- [86] Han Qiu, Tianwei Zhang, Tianzhu Zhang, Hongyu Li, and Meikang Qiu. 2023. DefQ: Defensive Quantization Against Inference Slow-Down Attack for Edge Computing. *IEEE Internet of Things Journal* 10, 4 (Feb. 2023), 3243–3252. doi:10.1109/JIOT.2021.3138935
- [87] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. 2021. DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Red Hook, NY, USA, 13937–13949. arXiv:2106.02034 doi:10.48550/arXiv.2106.02034
- [88] Ravishka Rathnasuriya, Tingxi Li, Zexin Xu, Zihe Song, Mirazul Haque, Simin Chen, and Wei Yang. 2025. SoK: Efficiency Robustness of Dynamic Deep Learning Systems. In *Proceedings of the 34th USENIX Security Symposium (USENIX Security)*. USENIX Association, Seattle, WA, USA, 4683–4702.
- [89] Ravishka Rathnasuriya and Wei Yang. 2025. Exploiting Efficiency Vulnerabilities in Dynamic Deep Learning Systems. arXiv preprint arXiv:2506.17621. doi:10.48550/arXiv.2506.17621
- [90] Yitong Ren, Qijie Li, Zhi Xiong, Wen Chen, and Run Zhao. 2025. DCT-based Fast Localization and Elimination of Adversarial Patches. In *International Conference on Real-time Computing and Robotics (RCAR)*. IEEE, Piscataway, NJ, USA, 1300–1307. doi:10.1109/RCAR65431.2025.11139457
- [91] Matthew Renze and Erhan Guven. 2024. The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models. arXiv preprint arXiv:2401.05618. doi:10.48550/arXiv.2401.05618
- [92] Debopam Sanyal, Jui-Tse Hung, Manavi Agrawal, Prahlad Jasti, Shahab Nikkhoo, Somesh Jha, Tianhao Wang, Sibin Mohan, and Alexey Tumanov. 2023. Pareto-Secure Machine Learning (PSML): Fingerprinting and Securing Inference Serving Systems. arXiv preprint arXiv:2307.01292. doi:10.48550/arXiv.2307.01292
- [93] Coen Schoof, Stefanos Koffas, Mauro Conti, and Stjepan Picek. 2024. Beyond PhantomSponges: Enhancing Sponge Attack on Object Detection Models. In *Proceedings of the 2024 ACM Workshop on Wireless Security and Machine Learning (WiseML)*. ACM, New York, NY, USA, 14–19. doi:10.1145/3649403.3656485
- [94] Avishag Shapira, Alon Zolfi, Luca Demetrio, Battista Biggio, and Asaf Shabtai. 2023. Phantom Sponges: Exploiting Non-Maximum Suppression to Attack Deep Object Detectors. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Piscataway, NJ, USA, 4571–4580. doi:10.1109/WACV56688.2023.00455
- [95] Ilia Shumailov, Yiren Zhao, Daniel Bates, Nicolas Papernot, Robert Mullins, and Ross Anderson. 2021. Sponge Examples: Energy-Latency Attacks on Neural Networks. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Piscataway, NJ, USA, 212–231. doi:10.1109/EuroSP51992.2021.00024
- [96] Manojna Sistla, Yu Wen, Aamir Bader Shah, Chenpei Huang, Lening Wang, Xuqing Wu, Jiefu Chen, Miao Pan, and Xin Fu. 2025. Bit-Flip Induced Latency Attacks in Object Detection. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, Piscataway, NJ, USA, 6709–6718. doi:10.1109/WACV61041.2025.00653
- [97] Tharun Adithya Srikrishnan, Deval Shah, Timothy Hein, Ahmed Hasssan, Stephen Youn, and Steven K. Reinhardt. 2025. BlindSight: Harnessing Sparsity for Efficient VLMs. arXiv preprint arXiv:2507.09071. doi:10.48550/arXiv.2507.09071
- [98] Jonathan Steinberg and Oren Gal. 2026. Semantic Denial of Service in LLM-Controlled Robots. arXiv preprint arXiv:2604.24790. doi:10.48550/arXiv.2604.24790
- [99] Zhaorui Sun, Zheng Zhang, Vishal Karande, Dave (Jing) Tian, and Ning Zhang. 2024. Side Channels via Speculative Decoding in LLMs. arXiv preprint arXiv:2411.01076. doi:10.48550/arXiv.2411.01076
- [100] Duoxun Tang, Dasen Dai, Jiyao Wang, Xiao Yang, Jianyu Wang, and Siqi Cai. 2026. VidDoS: Universal Denial-of-Service Attack on Video-based Large Language Models. arXiv preprint arXiv:2603.01454. doi:10.48550/arXiv.2603.01454

- [101] Jona te Lintelo, Stefanos Koffas, and Stjepan Picek. 2024. The SkipSponge Attack: Sponge Weight Poisoning of Deep Neural Networks. arXiv preprint arXiv:2402.06357. doi:10.48550/arXiv.2402.06357
- [102] Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2016. BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks. In *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR)*. IEEE, Piscataway, NJ, USA, 2464–2469. doi:10.1109/ICPR.2016.7900006
- [103] Kamala Varma, Arda Numanoglu, Yigitcan Kaya, and Tudor Dumitras. 2024. Understanding, Uncovering, and Mitigating the Causes of Inference Slowdown for Language Models. In *Proceedings of the IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, Piscataway, NJ, USA, 723–740.
- [104] Apostol Vassilev, Alina Oprea, Alie Fordyce, and Hyrum Anderson. 2023. *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*. Technical Report NIST AI 100-2e2023. National Institute of Standards and Technology. doi:10.6028/NIST.AI.100-2e2023
- [105] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. 2024. YOLOv10: Real-Time End-to-End Object Detection. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Red Hook, NY, USA, 107984–108011. arXiv:2405.14458 doi:10.48550/arXiv.2405.14458
- [106] Chenyi Wang, Ruoyu Song, Raymond Muller, Jean-Philippe Monteui, Z. Berkay Celik, Jonathan Petit, Ryan Gerdes, and Ming Li. 2026. CP-FREEZER: Latency Attacks against Vehicular Cooperative Perception. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 1114–1122. doi:10.1609/aaai.v40i2.37082
- [107] Derui Wang, Chaoran Li, Sheng Wen, Surya Nepal, and Yang Xiang. 2019. Daedalus: Breaking Non-Maximum Suppression in Object Detection via Adversarial Examples. arXiv:1902.02067 doi:10.48550/arXiv.1902.02067
- [108] Tianyi Wang, Huawei Fan, Yuanchao Shu, Peng Cheng, and Cong Wang. 2026. Rethinking Latency Denial-of-Service: Attacking the LLM Serving Framework, Not the Model. arXiv preprint arXiv:2602.07878. doi:10.48550/arXiv.2602.07878
- [109] Tianyi Wang, Zichen Wang, Cong Wang, Yuanchao Shu, Ruilong Deng, Peng Cheng, and Jiming Chen. 2025. Can't Slow Me Down: Learning Robust and Hardware-Adaptive Object Detectors against Latency Attacks for Edge Devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Piscataway, NJ, USA, 19230–19240. doi:10.1109/CVPR52734.2025.01791
- [110] Xin Wang, Jie Li, Zejia Weng, Yixu Wang, Yifeng Gao, Tianyu Pang, Chao Du, Yan Teng, Yingchun Wang, Zuxuan Wu, Xingjun Ma, and Yu-Gang Jiang. 2025. FreezeVLA: Action-Freezing Attacks against Vision-Language-Action Models. arXiv preprint arXiv:2509.19870. doi:10.48550/arXiv.2509.19870
- [111] Xi Wang, Anushri Suresh, Alvin Zhang, Rishi More, William Jurayj, Benjamin Van Durme, Mehrdad Farajtabar, Daniel Khashabi, and Eric Nalisnick. 2026. Conformal Thinking: Risk Control for Reasoning on a Compute Budget. arXiv preprint arXiv:2602.03814. doi:10.48550/arXiv.2602.03814
- [112] Xiaosi Wang, Tianliang Yao, Simin Chen, Runqi Wang, Lei Ye, Kuofeng Gao, Yi Huang, and Yuan Yao. 2025. VLMInfer-Slow: Evaluating the Efficiency Robustness of Large Vision-Language Models as a Service. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 16009–16024. doi:10.18653/v1/2025.acl-long.781
- [113] Yan Wang, Xingbin Wang, Zechao Lin, Yulan Su, Sisi Zhang, Rui Hou, and Dan Meng. 2024. Garrison: A High-Performance GPU-Accelerated Inference System for Adversarial Ensemble Defense. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. ACM, New York, NY, USA, 1–6. doi:10.1145/3649329.3654810
- [114] Yanlin Wang, Jiadong Wu, Tianyue Jiang, Mingwei Liu, Jiachi Chen, Chong Wang, Ensheng Shi, Xilin Liu, Yuchi Ma, and Zibin Zheng. 2026. DrainCode: Stealthy Energy Consumption Attacks on Retrieval-Augmented Code Generation via Context Poisoning. arXiv:2601.20615 doi:10.48550/arXiv.2601.20615
- [115] Zijian Wang, Shuo Huang, Yujin Huang, and Helei Cui. 2023. Energy-Latency Attacks to On-Device Neural Networks via Sponge Poisoning. In *Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Deep Learning Systems*. ACM, New York, NY, USA, 1–11. doi:10.1145/3591197.3591307
- [116] Lichen Xia, Xing Gao, and Weisong Shi. 2026. Investigating Stealthy Latency Attacks Against Camera-Based Object Detection Systems. In *Proceedings of the 4th USENIX Symposium on Vehicle Security and Privacy (VehicleSec)*. USENIX Association, Baltimore, MD, USA, 1–7.
- [117] Yong Xiao, Jin Ma, Ping Yi, and Xiuzhen Chen. 2024. Sponge Backdoor Attack: Increasing the Latency of Object Detection Exploiting Non-Maximum Suppression. In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, Piscataway, NJ, USA, 1–8. doi:10.1109/IJCNN60899.2024.10650435
- [118] Wenpeng Xing, Minghao Li, Mohan Li, and Meng Han. 2025. Towards Robust and Secure Embodied AI: A Survey on Vulnerabilities and Attacks. arXiv preprint arXiv:2502.13175. doi:10.48550/arXiv.2502.13175
- [119] Haochuan Xu, Yun Sing Koh, Shuhuai Huang, Zirun Zhou, Di Wang, Jun Sakuma, and Jingfeng Zhang. 2025. Model-agnostic Adversarial Attack and Defense for Vision-Language-Action Models. arXiv preprint arXiv:2510.13237. doi:10.48550/arXiv.2510.13237
- [120] Wenrui Xu and Keshab K. Parhi. 2025. A Survey of Attacks on Large Language Models. arXiv preprint arXiv:2505.12567. doi:10.48550/arXiv.2505.12567

- [121] Zhiyuan Xu, Joseph Gardiner, Sana Belguith, and Lichao Wu. 2026. RouteHijack: Routing-Aware Attack on Mixture-of-Experts LLMs. arXiv preprint arXiv:2605.02946. doi:10.48550/arXiv.2605.02946
- [122] Xiaobei Yan, Yiming Li, Hao Wang, Han Qiu, and Tianwei Zhang. 2025. BitHydra: Towards Bit-flip Inference Cost Attack against Large Language Models. arXiv preprint arXiv:2505.16670. doi:10.48550/arXiv.2505.16670
- [123] Oryan Yehezkel, Alon Zolfi, Amit Baras, Yuval Elovici, and Asaf Shabtai. 2024. DeSparsify: Adversarial Attack Against Token Sparsification Mechanisms in Vision Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates, Vancouver, Canada, 1–24. doi:10.48550/arXiv.2402.02554
- [124] Shengyao Zhang, Xudong Pan, Mi Zhang, and Min Yang. 2023. SlowBERT: Slow-down Attacks on Input-adaptive Multi-exit BERT. In *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, Stroudsburg, PA, USA, 9992–10007. doi:10.18653/v1/2023.findings-acl.634
- [125] Shengyao Zhang, Mi Zhang, Xudong Pan, and Min Yang. 2023. No-Skim: Towards Efficiency Robustness Evaluation on Skimming-based Language Models. arXiv preprint arXiv:2312.09494. doi:10.48550/arXiv.2312.09494
- [126] Xiaolei Zhang, Lu Zhou, Xiaogang Xu, Jiafei Wu, Tianyu Du, Heqing Huang, Hao Peng, and Zhe Liu. 2026. From Thinker to Society: Security in Hierarchical Autonomy Evolution of AI Agents. arXiv preprint arXiv:2603.07496. doi:10.48550/arXiv.2603.07496
- [127] Yan Zhang, Zihao Liu, Yi Zhu, and Chenglin Miao. 2025. Towards Real-Time Defense against Object-Based LiDAR Attacks in Autonomous Driving. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*. ACM, New York, NY, USA, 15 pages. doi:10.1145/3719027.3765227
- [128] Yuanhe Zhang, Xinyue Wang, Zhican Chen, Weiliu Wang, Zilu Zhang, Zhengshuo Gong, Zhenhong Zhou, Kun Wang, Li Sun, Yang Liu, and Sen Su. 2026. Resource Consumption Threats in Large Language Models. arXiv preprint arXiv:2603.16068. doi:10.48550/arXiv.2603.16068
- [129] Yuanhe Zhang, Xinyue Wang, Haoran Gao, Zhenhong Zhou, Fanyu Meng, Yuyao Zhang, and Sen Su. 2025. PD3F: A Pluggable and Dynamic DoS-Defense Framework against Resource Consumption Attacks Targeting Large Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Stroudsburg, PA, USA, 1–16. doi:10.48550/arXiv.2505.18680
- [130] Yuanhe Zhang, Zhenhong Zhou, Wei Zhang, Xinyue Wang, Xiaojun Jia, Yang Liu, and Sen Su. 2025. Crabs: Consuming Resource via Auto-generation for LLM-DoS Attack under Black-box Settings. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Stroudsburg, PA, USA, 11128–11150. doi:10.48550/arXiv.2412.13879
- [131] Zhihui Zhao, Xiaorong Dong, Yaowen Zheng, Xiaohui Chen, Yimo Ren, Hangbei Cheng, Yongle Chen, and Limin Sun. 2026. Breaking Cross-Modal Alignment in Embodied Intelligence: A Multimodal Adversarial Attack Framework for Vision-Language-Action Models. In *Proceedings of the ACM Web Conference 2026 (WWW)*. ACM, Sydney, Australia, 1–12. doi:10.1145/3774904.3792315
- [132] Kaiyu Zhou, Yongsen Zheng, Yicheng He, Meng Xue, Xueluan Gong, Yuji Wang, Xuanye Zhang, and Kwok-Yan Lam. 2026. Beyond Max Tokens: Stealthy Resource Amplification via Tool Calling Chains in LLM Agents. arXiv preprint arXiv:2601.10955. doi:10.48550/arXiv.2601.10955
- [133] Yuguang Zhou, Xunguang Wang, Pingchuan Ma, Zhantong Xue, Zhaoyu Wang, and Shuai Wang. 2026. From Shield to Target: Denial-of-Service Attacks on LLM-Based Agent Guardrails. arXiv preprint arXiv:2606.14517. doi:10.48550/arXiv.2606.14517
- [134] Zhenhong Zhou, Zherui Li, Jie Zhang, Yuanhe Zhang, Kun Wang, Yang Liu, and Qing Guo. 2026. CORBA: Contagious Recursive Blocking Attacks on Multi-Agent Systems Based on Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, CA, USA, 10 pages. doi:10.48550/arXiv.2502.14529
- [135] Zhenhao Zhu, Yue Liu, Zhiwei Xu, Yingwei Ma, Hongcheng Gao, Nuo Chen, Yanpei Guo, Wenjie Qu, Huiying Xu, Zifeng Kang, Xinzhong Zhu, and Jiaheng Zhang. 2025. ExtendAttack: Attacking Servers of LRMs via Extending Reasoning. arXiv preprint arXiv:2506.13737. doi:10.48550/arXiv.2506.13737

Appendix

A Glossary of Metrics and Terms

For quick reference, Table 7 collects the amplification metrics defined in Section 2.5 and the recurring acronyms used throughout the survey.

Table 7. Glossary of the amplification metrics (Section 2.5) and recurring acronyms.

Symbol / term	Meaning
<i>Amplification metrics (all relative to a matched benign reference; see Section 2.5)</i>	
$AF_M(x)$	Generic amplification factor: attack-induced metric $M(x)$ over the matched benign median $\text{median}_{x_0 \in \mathcal{B}(x)} M(x_0)$, where M may be work, latency, energy, or tokens. Report absolute increase when the benign median is zero/near-zero.
S_i, τ_i	Deadline slack $S_i = d_i - f_i$ and tardiness $\tau_i = \max(0, -S_i)$ for job i with deadline d_i and completion time f_i ; negative slack indicates a miss.
Throughput degradation	Relative degradation in throughput or goodput against a matched benign <i>work-load</i> (reported as a relative change, not a named symbol).
Zero rule	If a benign median denominator is 0, the ratio is undefined: report the absolute change and attack value, never an arbitrary ϵ .
$\mathcal{B}(x)$	Matched benign reference set for x (exact on model/config/hardware/concurrency; stratified on task and length; no post hoc matching on attacked values).
<i>Acronyms</i>	
AD	Autonomous driving.
DoS / DDoS	Denial of service / distributed denial of service.
DETR	DEtection TRansformer (query-based, NMS-free detector).
EOS	End-of-sequence token that terminates autoregressive generation.
FLOP	Floating-point operation (a proxy for computational work).
KV cache	Key-value cache storing past attention states in transformer decoding.
LLM / VLM	Large language model / vision-language model.
MoE	Mixture-of-experts (sparsely activated expert routing).
MOT	Multi-object tracking.
NMS	Non-maximum suppression (candidate-box filtering in detectors).
RAG	Retrieval-augmented generation.
TTFT	Time to first token (LLM-serving latency measure).
V2V	Vehicle-to-vehicle communication (cooperative perception).
VLA	Vision-language-action model (maps observations to robot actions).
WCET	Worst-case execution time.

B Catalog of Latency, Energy, and Timing Attacks and Defenses

This appendix provides a structured, citable catalog of the works surveyed, organized by attack stage (inference vs. training) and by defenses. Each entry records the target architecture, application domain, attacker setting, and links to the paper and released code where available. A continuously updated, searchable web version of these tables is maintained at the companion repository <https://github.com/guzonghua/awesome-latency-attacks>. Tables 8–10 mirror that resource.

Table 8. Inference-stage latency, energy, and timing attacks on deep learning.  links to the paper;  links to released code (**X** = none located). [†] = availability-adjacent (denies or halts service without demonstrated computational-work or latency amplification); [‡] = integrity attack with hypothesized but unevaluated latency implications (Section 4.5).

Attack	Venue (Year)	Target	Domain	Setting	Paper	Code
Daedalus	arXiv (2019)	Object detection (NMS)	CV / AD	White box (physical)		
Sparsity Attacks	IEEE TCAD (2020)	CNNs	CV	White box		X
ILFO	CVPR (2020)	Depth-adaptive networks (AdNNs)	CV	White box		X
GradAuto	ECCV (2022)	Depth- & width-dynamic AdNNs	CV	White box		
GradMDM	IEEE TPAMI (2023)	Dynamic neural networks	CV	White box		X
Tracker Hijacking	ICLR (2020)	Multi-object tracking	AD	White box		X
Sponge Examples	EuroS&P (2021)	Transformers + CNNs	CV & NLP	Black & white box		
DeepSloth	ICLR (2021)	Multi-exit CNNs	CV	White box		
AntiNODE	ICCVW (2023)	Neural ODEs (adaptive solver)	CV	Black box (transfer)		X
SlothSpeech	INTERSPEECH (2023)	Speech recognition (ASR)	Speech	White box		X
Timing Side-Channel AE	IEICE Trans. (2021)	DNN classifiers (timing oracle)	CV	Black box (timing)		X
SpikeAttack	ACM/IEEE DAC (2022)	Spiking neural networks	CV	White box		X
NMTSloth	ESEC/FSE (2022)	Decoder-based NMT	NLP	White box		
LLMEffiChecker	ACM TOSEM (2022)	LLMs	NLP	Black & white box		
NICGSlowDown	CVPR (2022)	Decoder-based image captioning	CV	White box		
SAME	ACL (2023)	Multi-exit transformers	NLP	White box		
SlowBERT	ACL Findings (2023)	Multi-exit BERT	NLP	White box		X
WAFFLE	NeurIPS (2023)	Multi-exit language models	NLP	White & black box		
DeepPerform	ASE (2022)	Dynamic (resource-constrained) DNNs	CV / NLP	White box (testing)		X
No-Skim	arXiv (2023)	Skimming language models	NLP	Black & white box		X
Phantom Sponges	WACV (2023)	Object detection (NMS)	CV / AD	White box		
SlowLiDAR	CVPR (2023)	3D LiDAR detection	CV / AD	White box		
WIP Tracker Attack	VehicleSec (2023)	Multi-object tracking	AD	White box (patch)		X
Variable-Time Inference	AISeC@CCS (2023)	Object detection (NMS timing)	CV	Black box (timing)		X
Overload	CVPR (2024)	Object detection (edge)	CV / AD	White box		X
SlowFormer	CVPR (2024)	Efficient vision transformers	CV	White box (patch)		
DeSparsify	NeurIPS (2024)	Token-sparsified ViTs	CV	White box		X
Beyond Phantom-Sponges	ACM WiseML (2024)	Object detection (NMS)	CV / AD	White box		X

continued on next page

Table 8. Inference-stage latency, energy, and timing attacks on deep learning.  links to the paper;  links to released code (X = none located). (continued)

Attack	Venue (Year)	Target	Domain	Setting	Paper	Code
SlowTrack	AAAI (2024)	Camera perception (de- tect+track)	AD	White box		
SlowPerception	arXiv (2024)	Camera perception (NMS+MOT)	AD	White box (physi- cal, projector)		X
TPA (Time-aware)	Pattern Recognition (2024)	Recog- Letters Detection + segmenta- tion	AD	White box		X
Steal Now Attack Later	arXiv (2024)	Object detection	CV	Black box		X
Energy Attack (multi- exit)	Info. & Software Tech. (2024)	Adaptive multi-exit net- works	CV	Grey box		X
Slowdown Causes (SaTML)	IEEE SaTML (2024)	Language models	NLP	White box		X
Engorgio	arXiv (2024)	LLMs (output inflation)	NLP	White box + trans- fer		X
Verbose Images	ICLR (2024)	Large VLMs	CV+NLP	White box		
Uniform Inputs	IEEE SPW (2024)	CNNs (sparsity)	CV	Black box		
Speculative- Decoding	arXiv (2024)	LLM serving (specula- tive decoding)	NLP	Black box (timing)		X
Side Channel						
DetStorm	IEEE S&P (2025)	Camera perception	AD	White box (physi- cal)		X
Inference-Time Impact Analysis	arXiv (2025)	Full perception (sim.)	AD	Simulation		X
DDLs Efficiency At- tacks	arXiv (2025)	Early-exit / token- pruning / MoE	CV & NLP	White & black box		X
TTSlow	IEEE TASLP (2025)	Auto-regressive TTS	Speech	White box		X
Crabs	ACL Findings (2025)	LLMs (DoS)	NLP	Black box		X
DDAS	NSF/edge (2022)	Edge dynamic (multi- exit) DNNs	CV	White box		X
EREBA	ICSE (2022)	Adaptive neural net- works (AdNNs)	CV	Black box		X
LoopLLM	AAAI (2026)	LLMs (repetitive gener- ation)	NLP	White box + trans- fer		X
RA-ICA (RAG)	ACM WWW (2026)	RAG-enhanced LLMs	NLP	Corpus poisoning		X
DrainCode	arXiv (2026)	RAG code generation	NLP / Code	Corpus poisoning		
VLMInferSlow	ACL (2025)	VLMs-as-a-service	CV+NLP	Black box		X
Verbose-Text Induc- tion	arXiv (2025)	VLMs	CV+NLP	White box		X
LingoLoop	arXiv (2025)	Multimodal LLMs	CV+NLP	White box		X
VidDoS	arXiv (2026)	Video-LLMs (streaming AD)	CV+NLP / AD	White box (univers- al patch)		X
FreezeVLA [†]	arXiv (2025)	Vision-language-action models	Robotics	White box (image)		X
Semantic-DoS [†]	arXiv (2026)	LLM-controlled robots	Robotics	Audio injection		X

continued on next page

Table 8. Inference-stage latency, energy, and timing attacks on deep learning.  links to the paper;  links to released code (X = none located). (continued)

Attack	Venue (Year)	Target	Domain	Setting	Paper	Code
EDPA [‡]	arXiv (2025)	Vision-language-action models	Robotics	Black box (patch)		
MAVLA [‡]	ACM WWW (2026)	Vision-language-action models	Robotics	White box		X
ANNIE [‡]	arXiv (2025)	Embodied AI / VLA robots	Robotics	White box		
Bit-Flip NMS Attack	WACV (2025)	Object detection (parameters)	CV / AD	Hardware (Rowhammer)		X
BitHydra	arXiv (2025)	LLM weights (no-EOS)	NLP	Hardware (bit-flip)		X
QuantAttack	WACV (2025)	Dynamically quantized ViTs	CV	White box		
Timestep-Compressed Attack	AAAI (2026)	Spiking neural networks	CV	White box		X
CP-FREEZER	AAAI (2026)	Cooperative perception (V2V)	AD	White box (testbed)		X
Trajectory-Aware Attack	IEEE TMM (2026)	Multi-object tracking	AD	White box		X
SPLAT	IEEE TCAD (2026)	Multi-exit dynamic networks	CV	Black box		X
RouteHijack [‡]	arXiv (2026)	Mixture-of-experts LLMs	NLP	White box		X
Misrouter	arXiv (2026)	Mixture-of-experts LLMs	NLP	Black box (input-only)		X
ReasoningBomb	ACM CCS (2026)	Large reasoning models	NLP	Black box		X
ThinkTrap	NDSS (2026)	Reasoning LLM APIs	NLP	Black box		X
OverThink	ICML (2025)	Reasoning LLMs (RAG/context)	NLP	Indirect injection		X
ExtendAttack	arXiv (2025)	Large reasoning models	NLP	Black box (encoding)		X
RepetitionCurse	arXiv (2025)	MoE LLMs (expert parallelism)	NLP	Black box		X
Fill and Squeeze	arXiv (2026)	LLM serving scheduler	NLP	Black box (system)		X
Beyond Max Tokens	arXiv (2026)	LLM agent tool chains (MCP)	NLP / Agentic	Black box		X
From Shield to Target	arXiv (2026)	LLM agent guardrails	NLP / Agentic	Black box (transfer)		X
Mobius Injection (ABo-DDoS)	arXiv (2026)	LLM agent infrastructure	Agentic	Injection		X
CORBA	ACL Findings (2026)	LLM multi-agent systems	Agentic	Contagious injection		

Table 9. Training-stage (poisoning / backdoor / weight) latency and energy attacks.

Attack	Venue (Year)	Target	Domain	Setting	Paper	Code
Sponge Poisoning	arXiv / AsiaCCS (2022)	CNNs	CV	Partial control		

continued on next page

Table 9. Training-stage (poisoning / backdoor / weight) latency and energy attacks. (continued)

Attack	Venue (Year)	Target	Domain	Setting	Paper	Code
On-Device Poisoning	Sponge ACM SecTL (2023)	On-device DNNs	CV	Partial control		✗
Mobile-App	Sponge ACM HotMobile (2023)	Mobile ML models	CV	Partial control		✗
SkipSponge	arXiv (2024)	CNNs, GANs (weights)	CV	Full control		✗
Huang et al. (multi-exit)	IEEE Access (2024)	Multi-exit CNNs	CV	Full control		✗
Sponge Backdoor (OD)	IJCNN (2024)	Object detection (NMS)	CV / AD	Backdoor		✗
DoS Poisoning (LLM)	arXiv (2024)	LLMs (no-EOS)	NLP	Backdoor / poisoning		✗
Sensing-AI Sponge	IEEE GLOBECOM (2025)	Sensing DNNs (IoT)	Sensing	Partial control		✗
EvoWeight (FPGA)	IEEE HOST (2025)	FPGA DNN accelerators	CV	Full control		✗
Reflection Backdoor (VLM-AD)	arXiv (2025)	Driving VLM planner	AD	Backdoor (physical trigger)		✗

Table 10. Defenses against latency, energy, and timing attacks, by control mechanism.

Defense	Venue (Year)	Target	Mechanism	Domain	Paper	Code
DEE Scheduling	ACM CIKM (2021)	Early-exit networks	Runtime scheduling	CV		✗
Certifier Caveat	arXiv (2021)	Certified classifiers	Defense pitfall	CV		✗
Constant-Time NMS	AISeC@CCS (2023)	Object detection (NMS)	Constant-time implementation (within analyzed NMS boundary)	CV		✗
PSML	arXiv (2023)	Inference serving systems	System / serving control	ML serving		✗
Adaptive Resizing	ACM WiseML (2024)	Object detection (NMS)	Input transformation	AD		✗
ADAV Patch Defense	arXiv (2024)	Object detection	Input transformation	AD		✗
Securing AV Perception	IEEE TIV (2024)	AV visual perception	Input transformation	AD		✗
Calibration Defense	IEEE SaTML (2024)	Multi-exit models	Robust training	NLP		✗
DefQ	IEEE IoT-J (2023)	Multi-exit DNNs (DeepSloth)	DCT defensive quantization	Edge		✗
Feature Distillation	CVPR (2019)	CNNs (JPEG pipeline)	DCT input purification	CV		✗
Sparsity Monitor	IEEE SPW (2024)	CNNs	Runtime monitoring	CV		
Garrison	ACM/IEEE DAC (2024)	Ensemble inference (GPU)	System / serving control	CV		✗
Time-Traveling Defense	arXiv (2024)	Traffic-sign classifiers	Temporal redundancy	AD		✗
Can't Slow Me Down	CVPR (2025)	Edge object detectors	Robust/adaptive training	AD		✗
DCT Patch Elimination	IEEE RCAR (2025)	Object detection	Input transformation	AD		✗

continued on next page

Table 10. Defenses against latency, energy, and timing attacks, by control mechanism. (continued)

Defense	Venue (Year)	Target	Mechanism	Domain	Paper	Code
Real-Time LiDAR Defense	ACM CCS (2025)	LiDAR detection	Runtime monitoring	AD		
LDP Purification	IEEE TrustCom (2025)	VLM visual encoders	Input purification	CV+NLP		
Pruning Defense	IEEE GLOBECOM (2025)	Sensing DNNs	Architectural sparsity	Sensing		
BlindSight	arXiv (2025)	VLMs	Architectural sparsity	CV+NLP		
PD3F	EMNLP (2025)	LLM serving	Runtime scheduling and termination	NLP		
SQUAD	arXiv (2026)	Early-exit ensembles	Runtime scheduling	CV		
Token-Budget Routing	arXiv (2026)	LLM serving	Budget estimation and pool routing	NLP		
TALE (Token-Budget Reasoning)	arXiv (2024)	Reasoning LLMs	Prompt-based budget guidance	NLP		
Concise CoT (CCoT)	arXiv (2024)	Reasoning LLMs	Prompt-based concision guidance	NLP		
CoT-Valve	ACL (2025)	Reasoning LLMs	Learned/parameter-level length control	NLP		
Conformal Thinking	arXiv (2026)	Reasoning models	Risk-controlled adaptive stopping	NLP		

C Attack-Mapping Matrix

Table 11 maps representative attacks across five dimensions used throughout the survey: the attacker *interface* (Section 3), the exploited *bottleneck*, the *work measure* the attacker inflates, the *system consequence*, and the strongest *evidence setting* reported. The 18 entries were selected for coverage—at least one representative per interface (physical, API, corpus, hardware) and per major bottleneck (NMS, tracking, cooperative fusion, activation sparsity, early exit, token pruning, MoE routing, autoregressive decode, reasoning length, serving scheduler, agent graph, weight tampering)—rather than by highest reported slowdown. The matrix is a quick cross-index, not a quantitative comparison; reported magnitudes and their caveats remain in Tables 3, 4, and the catalog above; per-entry publication status is given in the catalog (Tables 8–10).

Table 11. Attack-mapping matrix. Interface: Phys. = physical sensor delivery; API = request/prompt; Corpus = retrieval/training data; HW = hardware fault. Evidence: Sim = digital model evaluation; HW = hardware-timed evaluation; Phys = physical sensor delivery; Test = vehicle/robot testbed; Loop = closed-loop simulator; Fault = hardware fault model (per Table 3). Per-entry publication status is listed in the catalog (Tables 8–10).

Attack	Interface	Bottleneck	Work measure	Consequence	Evid.
Phantom Sponges [94]	Phys./digital	NMS	Candidate boxes	OD latency	Sim
Groundswell [116]	Phys.	NMS	Candidate boxes	OD latency	Phys
SlowTrack [72]	Phys./digital	NMS + MOT	Boxes; track assoc.	Crash (sim.)	Loop
SlowPerception [73]	Phys. (proj.)	NMS + MOT	Phantom objects	Collision (sim.)	Phys
CP-FREEZER [106]	V2V message	Coop. fusion	Fused entities	Fusion delay	Test
SlowLiDAR [64]	Digital	LiDAR detect.	Points/candidates	Detect. latency	Sim
Sponge examples [95]	Digital	Activ. sparsity	Active neurons	Energy/latency	HW
DeepSloth [42]	Digital	Early exit	Executed layers	Slowdown	Sim
SlowFormer [80]	Phys. (patch)	Token pruning	Retained tokens	Compute inflation	Sim
RepetitionCurse [46]	API	MoE routing	Expert load	Serving latency	HW
NMTSloth [13]	API	AR decode	Decoder steps	NMT latency	Sim
Engorgio [20]	API	AR decode	Output tokens	LLM cost	Sim
ReasoningBomb [66]	API	Reasoning length	CoT tokens	LLM cost	Sim
OverThink [55]	Corpus (RAG)	Reasoning length	CoT tokens	LLM cost	Sim
DrainCode [114]	Corpus (RAG)	AR decode	Output tokens	Code-LLM cost	Sim
Fill and Squeeze [108]	API	Sched. / KV cache	KV-cache occ.	TTFT / HoL block	HW
Mobius Injection [61]	API (agentic)	Agent graph	Recursive calls	Cluster-wide DoS	HW
BitHydra [122]	HW (bit-flip)	Weights (no-EOS)	Output tokens	Persistent cost	Fault